

## 3- ARM CORTEX-M ARCHITECTURE

---

- History of ARM
- Cortex-M Features
- Cortex-M Instruction Set Architecture
- Instruction Set
- Memory Access
  - Memory-mapped I/O
- Exception Handling 

## 3.5 - EXCEPTIONS

---

The normal flow of execution of a program is to execute the next instruction in memory, unless a Branch, Subroutine Call or Return is executed. Hence, a program is executed in the same order.

**Exceptions are events that cause changes in program flow control outside a normal programmed sequence.**

An exception is a break in the normal flow of execution. It can be caused by:

Although, Faults are usually due to error conditions HW Interrupts and SW exceptions are normal in a program execution.

- Hardware interrupt
- Fault (e.g. memory access error)
- Software generated

EXCEPTIONS occur normally in Embedded Systems

## 3.5 - EXCEPTIONS

---

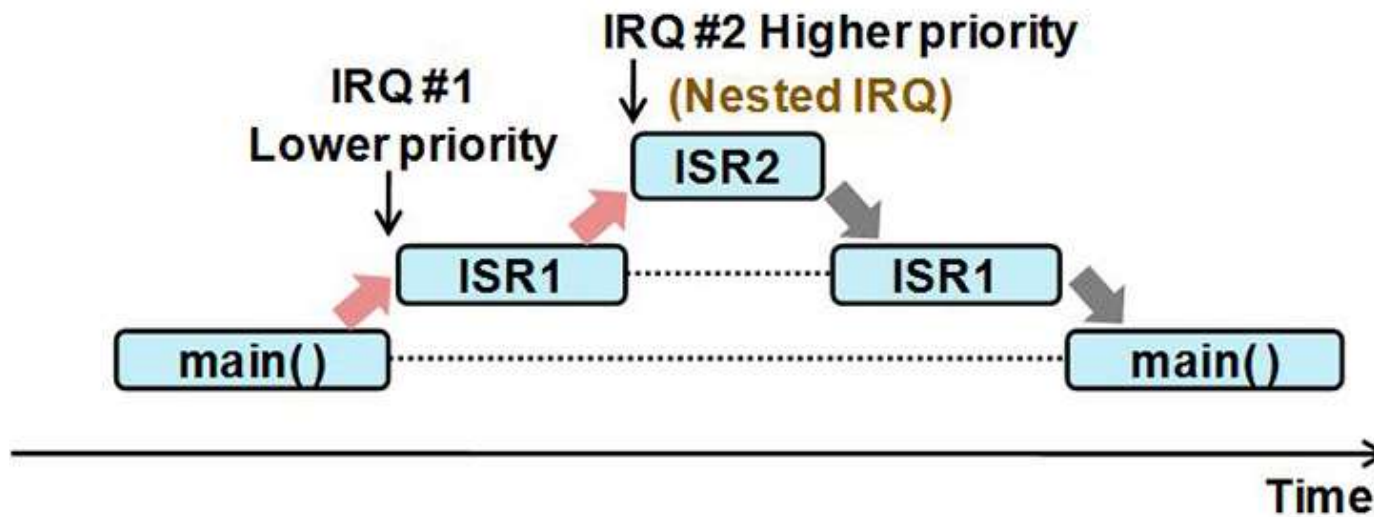
Exceptions occur **asynchronously**, this is, at any point of the execution. They may occur many times and on successive executions of the program they usually occur at different places of this program.

When an exception occurs, it must be **serviced**. Meaning that a software routine must either respond to the interrupt request or take steps to resolve or mitigate the fault.

This routine is called: **exception handler routine, interrupt service routine, or interrupt handler.**

# PREEMPTION

---



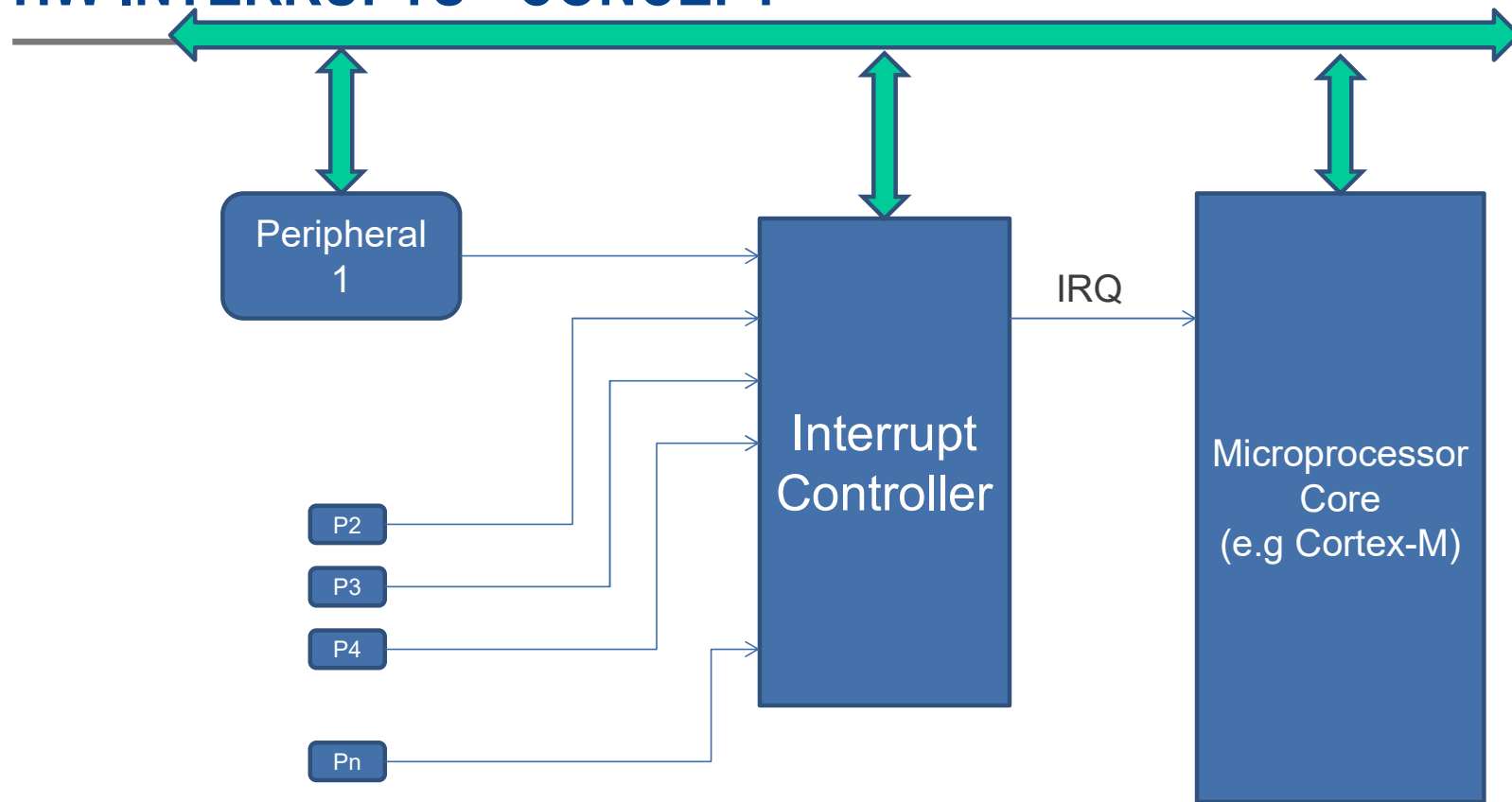
## 3.5 - EXCEPTIONS - INTERRUPTS

---

Hardware Interrupts are one of the kind of exceptions.

Interrupts are an **efficient** way for a peripheral to inform the processor that it requires servicing. If interrupts were not available, the processor would have to periodically poll the peripherals (thus termed **polling**) to check if service is required. Polling is an inefficient technique.

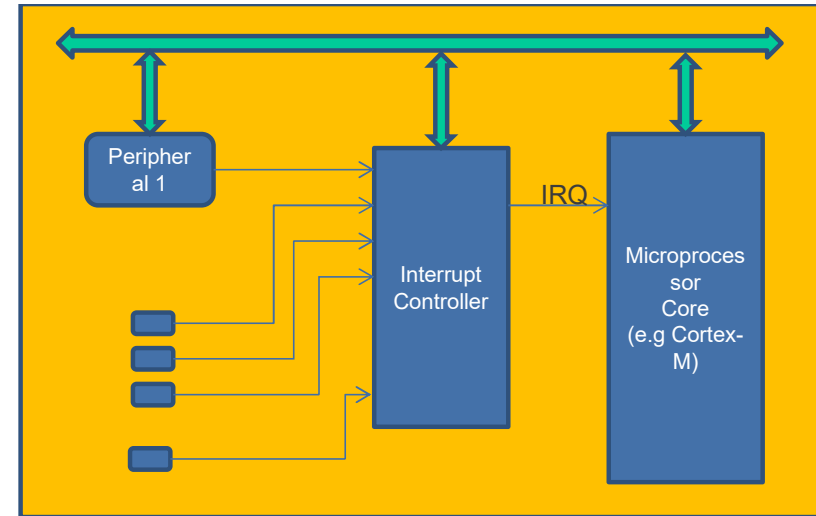
# HW INTERRUPTS - CONCEPT



## 3.5 - EXCEPTIONS - INTERRUPTS

When a peripheral or hardware needs service from the processor, typically the following sequence would occur:

1. The peripheral asserts an interrupt request to the processor
2. The processor suspends the task currently executing
3. The processor executes an Interrupt Service Routine (ISR) to service the peripheral, and, if needed, clears the interrupt request by software
4. The processor resumes the previously suspended task



# VECTORED INTERRUPT CONTROLLER

Which peripheral requested the interrupt ?

## Non-vectored:

A single (system) ISR consults the Interrupt Controller to identify the source of the interrupt.

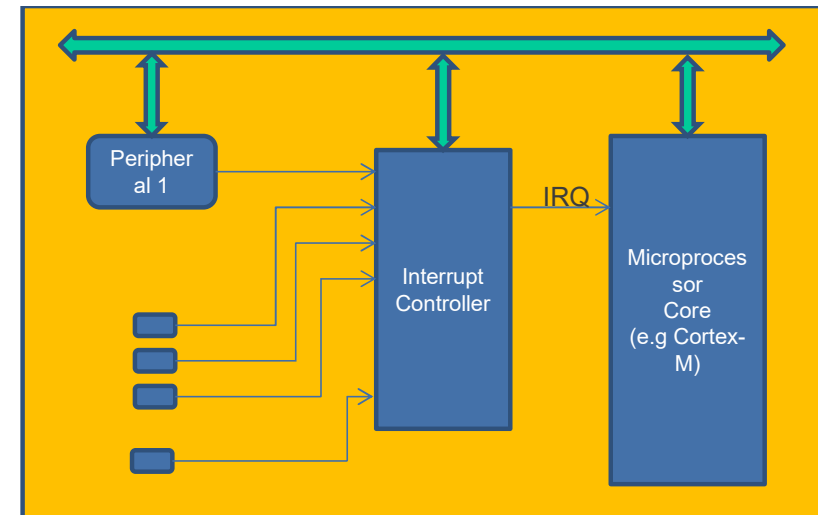
=> OVERHEAD

=> single ISR

## Vectored:

The interrupt controller informs the processor of the handler's address.

Cortex-M4: table of handler's address starting at address 0x0.



# INTERRUPTS - CONCEPTS

---

## **Latency:**

- time from IRQ request (signal) to start of handler
- encompasses:
  - detection,
  - processing in the IRQ Controller,
  - finish current instruction,
  - period that IRQs are masked,
  - higher priority handlers in execution (device drivers, RTOS, other threads)

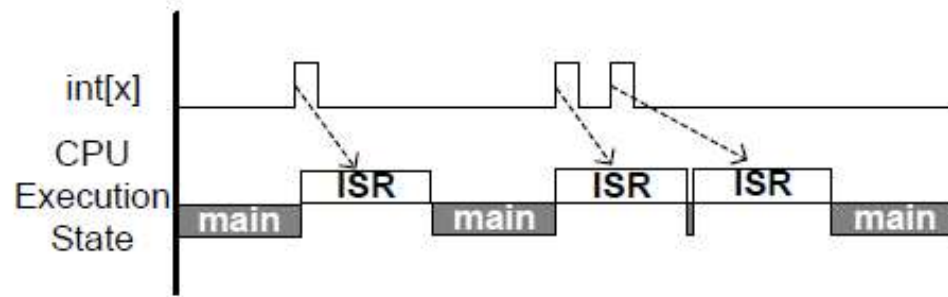
## **Response Time (worst case):**

- add: highest latency + worst case execution time of handler (ISR)

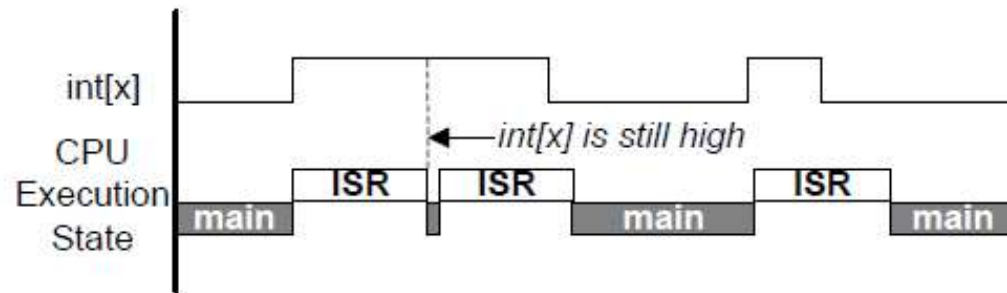
Cortex-M4 accepts both cases, no need to configure.  
Pending status is cleared automatically when handler is started.

## LEVEL VS EDGE INTERRUPTS

Edge



Level



# EXCEPTION AND INTERRUPT NUMBERING

- Exceptions are numbered from 1 to 255
- 1..15 - system exceptions
- 16 .. - IRQ0 to IRQ 239

Attention to difference of 16 between exception and interrupt numbering.

The exception number is registered in IPSR

| Exception number <sup>a</sup> | IRQ number <sup>a</sup> | Exception type  | Priority                  |
|-------------------------------|-------------------------|-----------------|---------------------------|
| 1                             | -                       | Reset           | -3, the highest           |
| 2                             | -14                     | NMI             | -2                        |
| 3                             | -13                     | HardFault       | -1                        |
| 4                             | -12                     | MemManage       | Configurable <sup>c</sup> |
| 5                             | -11                     | BusFault        | Configurable <sup>c</sup> |
| 6                             | -10                     | UsageFault      | Configurable <sup>c</sup> |
| 7-10                          | -                       | Reserved        | -                         |
| 11                            | -5                      | SVCall          | Configurable <sup>c</sup> |
| 12-13                         | -                       | Reserved        | -                         |
| 14                            | -2                      | PendSV          | Configurable <sup>c</sup> |
| 15                            | -1                      | SysTick         | Configurable <sup>c</sup> |
| 16                            | 0                       | Interrupt (IRQ) | Configurable <sup>d</sup> |

# VECTOR TABLE

LSbit set to 1 - Thumb state

| Memory Address |                      | Exception Number |
|----------------|----------------------|------------------|
|                |                      |                  |
|                |                      |                  |
| 0x0000004C     | Interrupt#3 vector   | 19               |
| 0x00000048     | Interrupt#2 vector   | 18               |
| 0x00000044     | Interrupt#1 vector   | 17               |
| 0x00000040     | Interrupt#0 vector   | 16               |
| 0x0000003C     | SysTick vector       | 15               |
| 0x00000038     | PendSV vector        | 14               |
| 0x00000034     | Not used             | 13               |
| 0x00000030     | Debug Monitor vector | 12               |
| 0x0000002C     | SVC vector           | 11               |
| 0x00000028     | Not used             | 10               |
| 0x00000024     | Not used             | 9                |
| 0x00000020     | Not used             | 8                |
| 0x0000001C     | Not used             | 7                |
| 0x00000018     | Usage Fault vector   | 6                |
| 0x00000014     | Bus Fault vector     | 5                |
| 0x00000010     | MemManage vector     | 4                |
| 0x0000000C     | HardFault vector     | 3                |
| 0x00000008     | NMI vector           | 2                |
| 0x00000004     | Reset vector         | 1                |
| 0x00000000     | MSP initial value    | 0                |

Note : LSB of each vector must be set to 1 to indicate Thumb state

# FAULTS AND DEBUG EXCEPTIONS

---

**Hard Fault** - bus error on a vector read.

**Bus error** - usually occurs when AHB interface receives an error response from a bus slave (also called prefetch abort if it is an instruction fetch or data abort if it is a data access). Can also be caused by other illegal accesses.

**Memory management fault** - MPU violation or program execution from address locations with XN (eXecute Never) memory attribute.

**Usage Fault** - Exceptions due to program error -  
e.g. invalid instruction.

**Debug Monitor** - exception for debug events like breakpoints, watchpoints when software based debug solution is used.

---

## FAULTS AND DEBUG EXCEPTIONS

---

|                                         |           |
|-----------------------------------------|-----------|
| Bus error on a vector read error        | HardFault |
| Fault that is escalated to a hard fault |           |
| Fault on breakpoint escalation          |           |

|                                                        |           |
|--------------------------------------------------------|-----------|
| Fault on instruction access                            | MemManage |
| Fault on direct data access                            |           |
| Context stacking, because of an MPU access violation   |           |
| Context unstacking, because of an MPU access violation |           |
| During lazy floating-point state preservation          |           |

# FAULTS AND DEBUG EXCEPTIONS

|                                               |          |
|-----------------------------------------------|----------|
| During exception stacking                     | BusFault |
| During exception unstacking                   |          |
| During instruction prefetching, precise       |          |
| During lazy floating-point state preservation |          |
| Precise data access error, precise            |          |
| Imprecise data access error, imprecise        |          |

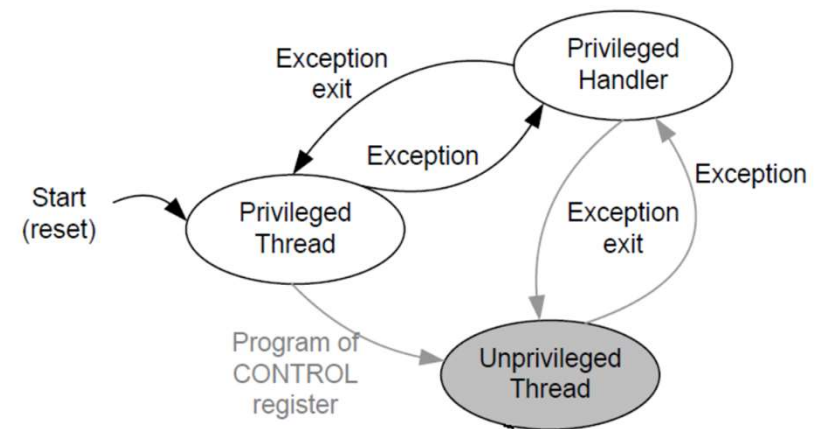
|                                                   |            |
|---------------------------------------------------|------------|
| Undefined instruction                             | UsageFault |
| Attempt to enter an invalid instruction set state |            |
| Failed integrity check on exception return        |            |
| Attempt to access a non-existing coprocessor      |            |
| Illegal unaligned load or store                   |            |
| Stack overflow                                    |            |
| Divide By 0                                       |            |

# SW EXCEPTIONS

---

**SVC Programmable SuperVisor Call** - usually used in OS environment to allow application tasks to access system services.

**PendSV - Pendable service call** - An exception usually used by an OS in processes like context switching.



# EXCEPTION PRIORITY

Higher priority preempts lower priority, i.e. can interrupt the execution of its handler.

**Higher** numbers => **lower** priority

Fixed priorities: -3, -2, -1

Programmable priorities: 0..255

(\*) number of priorities is implementation dependent.

| Exception number <sup>a</sup> | IRQ number <sup>a</sup> | Exception type  | Priority                  |
|-------------------------------|-------------------------|-----------------|---------------------------|
| 1                             | -                       | Reset           | -3, the highest           |
| 2                             | -14                     | NMI             | -2                        |
| 3                             | -13                     | HardFault       | -1                        |
| 4                             | -12                     | MemManage       | Configurable <sup>c</sup> |
| 5                             | -11                     | BusFault        | Configurable <sup>c</sup> |
| 6                             | -10                     | UsageFault      | Configurable <sup>c</sup> |
| 7-10                          | -                       | Reserved        | -                         |
| 11                            | -5                      | SVCall          | Configurable <sup>c</sup> |
| 12-13                         | -                       | Reserved        | -                         |
| 14                            | -2                      | PendSV          | Configurable <sup>c</sup> |
| 15                            | -1                      | SysTick         | Configurable <sup>c</sup> |
| 16                            | 0                       | Interrupt (IRQ) | Configurable <sup>d</sup> |

# CMSIS

---

```
/* ----- Interrupt Number Definition ----- */

typedef enum {
/* ----- Cortex-M4 Processor Exceptions Numbers ----- */
    Reset_IRQn                = -15,          /*!<  1  Reset Vector, invoked on Power up and warm reset           */
    NonMaskableInt_IRQn       = -14,          /*!<  2  Non maskable Interrupt, cannot be stopped or preempted     */
    HardFault_IRQn            = -13,          /*!<  3  Hard Fault, all classes of Fault                            */
    MemoryManagement_IRQn     = -12,          /*!<  4  Memory Management, MPU mismatch, including Access Violation
                                                and No Match                                                         */
    BusFault_IRQn             = -11,          /*!<  5  Bus Fault, Pre-Fetch-, Memory Access Fault, other address/memory
                                                related Fault                                                         */
    UsageFault_IRQn           = -10,          /*!<  6  Usage Fault, i.e. Undef Instruction, Illegal State Transition */
    SVCall_IRQn               =  -5,          /*!< 11  System Service Call via SVC instruction                       */
    DebugMonitor_IRQn         =  -4,          /*!< 12  Debug Monitor                                                 */
    PendSV_IRQn               =  -2,          /*!< 14  Pendable request for system service                         */
    SysTick_IRQn              =  -1,          /*!< 15  System Tick Timer                                             */
} IRQn_Type;
```

**// file S7G2.h**

---

# EXCEPTION HANDLERS

## Types of Exception Handlers:

**Interrupt Service Routines (ISRs)** — ISRs handle Interrupts IRQ0 to IRQxx.

**Fault handlers** — HardFault, MemManage fault, UsageFault, and BusFault are fault exceptions handled by the fault handlers.

**System handlers** — Handle: NMI, PendSV, SVCall SysTick, and fault exceptions. Hence, Fault Handlers are a subset of System Handlers.

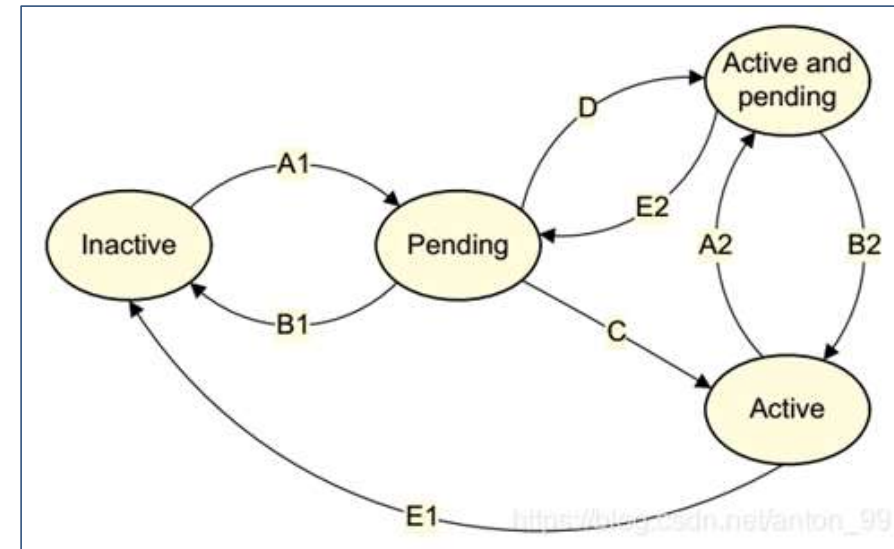
## EXCEPTION STATES

**Inactive** — The exception is not active and not pending.

**Pending** — The exception is waiting to be serviced by the processor. An interrupt request from a peripheral or from software can change the state of the corresponding interrupt to pending.

**Active** — An exception that is being serviced by the processor but has not completed. An exception handler can interrupt the execution of another exception handler. In this case both exceptions are in the active state.

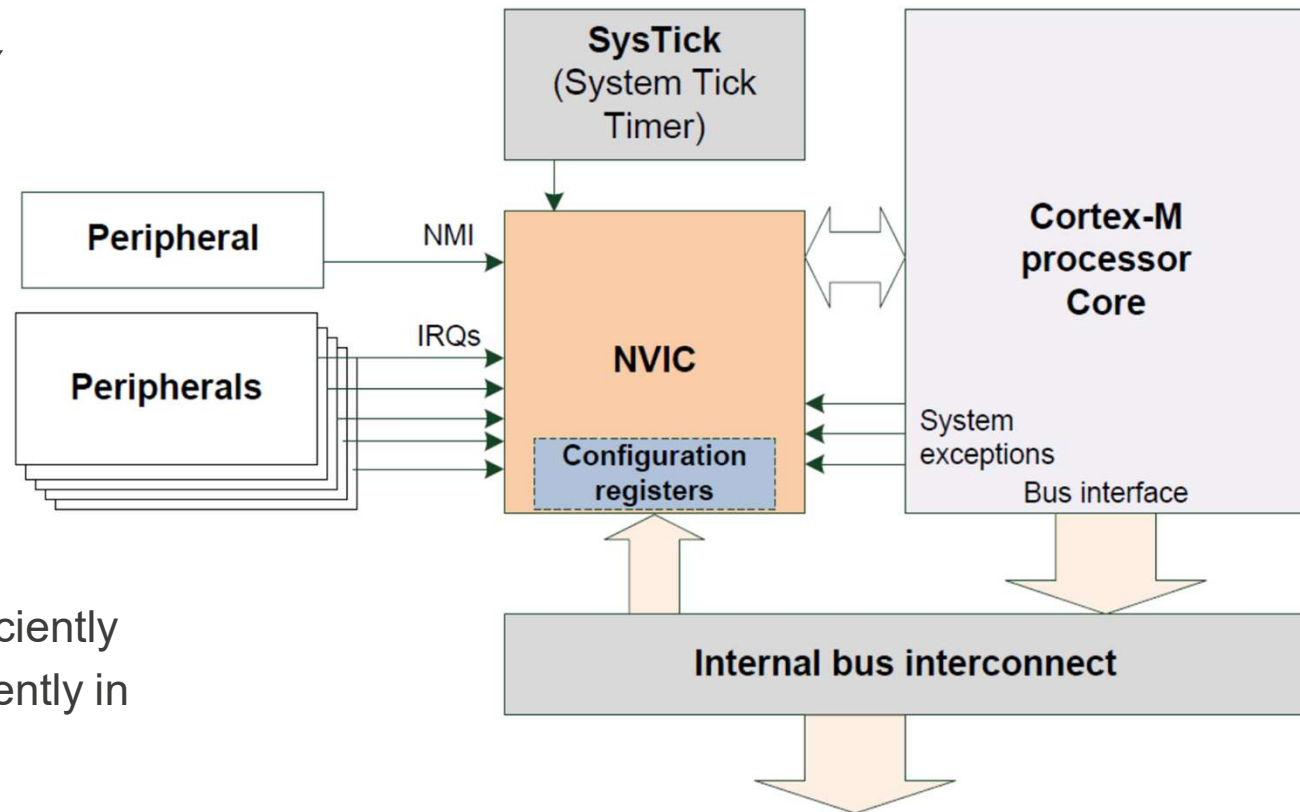
**Active and pending** — The exception is being serviced by the processor and there is a pending exception from the same source.



## 3.5 - EXCEPTIONS - INTERRUPTS

### Basic concepts of interrupt ENTRY

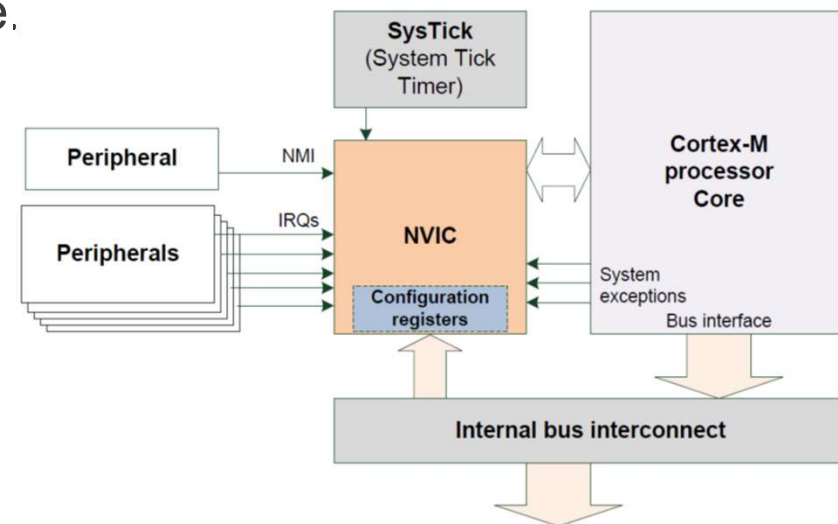
1. Peripheral sends an Interrupt Request (IRQ) to an Interrupt Controller
2. Interrupt Controller selects the highest priority non-masked interrupt request and informs the core.
3. If the priority of the IRQ is sufficiently high, when the instruction currently in execution finishes then the IRQ is serviced.



## 3.5 - EXCEPTIONS - INTERRUPTS - DETAILED PROCESS

1- An external device, such as a peripheral, requests an interrupt (IRQ) by signaling to the interrupt controller.

The input lines of the interrupt controller (240 in the NVIC of a Cortex-M4) can be either level-sensitive or edge-sensitive.

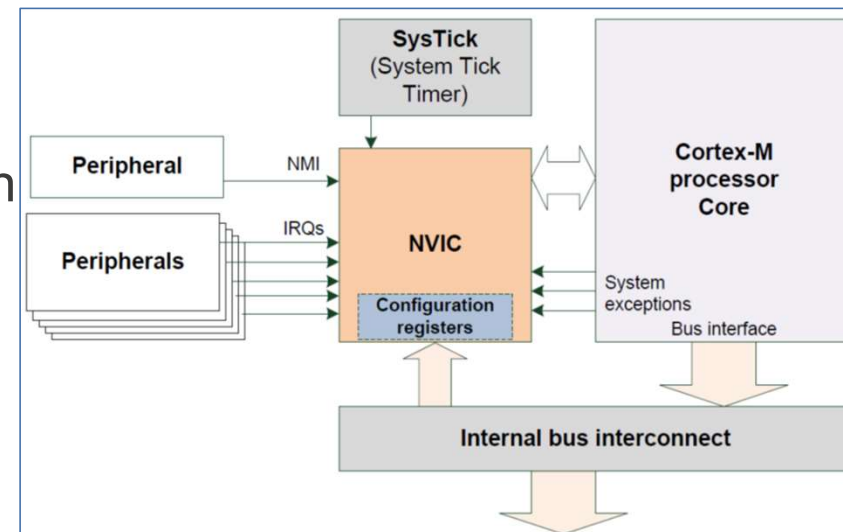


## 3.5 - EXCEPTIONS - INTERRUPTS - DETAILED PROCESS

2- Upon receiving an IRQ<sub>i</sub> (hardware signal on input *i* of the interrupt controller - IC) then the IC performs two checks:

- a) if input *i* is masked or not;
- b) if there is another request (IRQ<sub>j</sub> on input *j*) already being sent to the processor.

**If** IRQ<sub>i</sub> is not masked and if its priority is higher than IRQ<sub>j</sub>'s priority (or no request is currently being sent to the processor)  
**then** IRQ<sub>i</sub> is forwarded to the processor.



## 3.5 - EXCEPTIONS - INTERRUPTS - DETAILED PROCESS

3- The processor, upon receiving an IRQ verifies if its priority is sufficiently high:

- a) PRIMASK and FAULTMASK, when set, impose a priority level of 0 or -1 respectively. Hence, when FAULTMASK is set, all exceptions from 3 on are masked. Also, check if the requesting IRQ priority is higher than BASEPRI.
- b) If an exception is active (being serviced) then only a higher priority exception may preempt its handler.

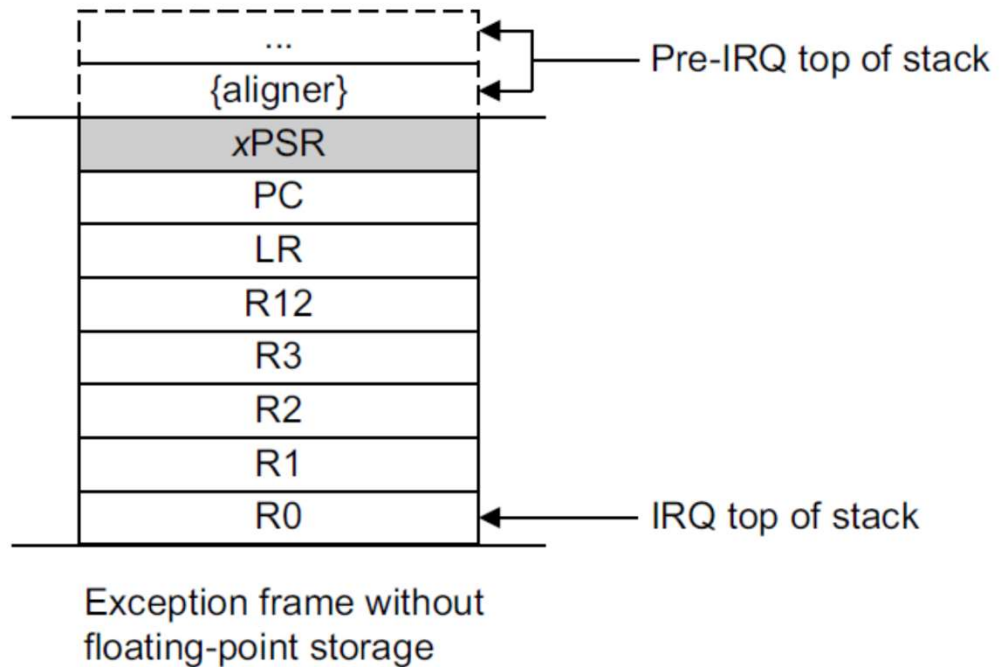
If both these conditions are met, servicing starts at the end of the current instruction.

| Exception number <sup>a</sup> | IRQ number <sup>a</sup> | Exception type  | Priority                  | Vector address or offset <sup>b</sup> | Activation                                            |
|-------------------------------|-------------------------|-----------------|---------------------------|---------------------------------------|-------------------------------------------------------|
| 1                             | -                       | Reset           | -3, the highest           | 0x00000004                            | Asynchronous                                          |
| 2                             | -14                     | NMI             | -2                        | 0x00000008                            | Asynchronous                                          |
| 3                             | -13                     | HardFault       | -1                        | 0x0000000C                            | -                                                     |
| 4                             | -12                     | MemManage       | Configurable <sup>c</sup> | 0x00000010                            | Synchronous                                           |
| 5                             | -11                     | BusFault        | Configurable <sup>c</sup> | 0x00000014                            | Synchronous when precise, asynchronous when imprecise |
| 6                             | -10                     | UsageFault      | Configurable <sup>c</sup> | 0x00000018                            | Synchronous                                           |
| 7-10                          | -                       | Reserved        | -                         | -                                     | -                                                     |
| 11                            | -5                      | SVCall          | Configurable <sup>c</sup> | 0x0000002C                            | Synchronous                                           |
| 12-13                         | -                       | Reserved        | -                         | -                                     | -                                                     |
| 14                            | -2                      | PendSV          | Configurable <sup>c</sup> | 0x00000038                            | Asynchronous                                          |
| 15                            | -1                      | SysTick         | Configurable <sup>c</sup> | 0x0000003C                            | Asynchronous                                          |
| 16                            | 0                       | Interrupt (IRQ) | Configurable <sup>d</sup> | 0x00000040 <sup>e</sup>               | Asynchronous                                          |

## 3.5 - EXCEPTIONS - INTERRUPTS - DETAILED PROCESS

4- Eight registers are pushed onto the Stack  
The Interrupt changes state to Active.

Since R0-R3 and R12 are stacked,  
any C procedure following AAPCS  
can be registered as a handler.



## 3.5 - EXCEPTIONS - INTERRUPTS - DETAILED PROCESS

5- Register LR is loaded with one of the EXC\_RETURN values, depending on the current state of the processor.

Note that EXC\_RETURN represents memory addresses in a region where code is not allowed.

| EXC_RETURN[31:0] | Description                                                                                                              |
|------------------|--------------------------------------------------------------------------------------------------------------------------|
| 0xFFFFFFF1       | Return to Handler mode, exception return uses non-floating-point state from the MSP and execution uses MSP after return. |
| 0xFFFFFFF9       | Return to Thread mode, exception return uses non-floating-point state from MSP and execution uses MSP after return.      |
| 0xFFFFFFF0       | Return to Thread mode, exception return uses non-floating-point state from the PSP and execution uses PSP after return.  |
| 0xFFFFFFE1       | Return to Handler mode, exception return uses floating-point-state from MSP and execution uses MSP after return.         |
| 0xFFFFFE9        | Return to Thread mode, exception return uses floating-point state from MSP and execution uses MSP after return.          |
| 0xFFFFFED        | Return to Thread mode, exception return uses floating-point state from PSP and execution uses PSP after return.          |

## 3.5 - EXCEPTIONS - INTERRUPTS - DETAILED PROCESS

6- The processor reads from the vector table the initial address of the handler for the Interrupt. This value is loaded to PC and the execution of the handler starts.

*Important:* since the handler is Thumb-2 code the addresses in the vector table must have its LSb set to 1.

| Exception number | IRQ number | Offset    | Vector                  |
|------------------|------------|-----------|-------------------------|
| 16+n             | n          | 0x0040+4n | IRQn                    |
| .                | .          | .         | .                       |
| .                | .          | .         | .                       |
| .                | .          | .         | .                       |
| 18               | 2          | 0x004C    | IRQ2                    |
| 17               | 1          | 0x0048    | IRQ1                    |
| 16               | 0          | 0x0044    | IRQ0                    |
| 15               | -1         | 0x0040    | Systick                 |
| 14               | -2         | 0x003C    | PendSV                  |
| 13               |            | 0x0038    | Reserved                |
| 12               |            |           | Reserved for Debug      |
| 11               | -5         | 0x002C    | SVCall                  |
| 10               |            |           | Reserved                |
| 9                |            |           |                         |
| 8                |            |           |                         |
| 7                |            |           | Reserved                |
| 6                | -10        |           |                         |
| 5                | -11        | 0x0018    |                         |
| 4                | -12        | 0x0014    | Usage fault             |
| 3                | -13        | 0x0010    | Bus fault               |
| 2                | -14        | 0x000C    | Memory management fault |
| 1                |            | 0x0008    | Hard fault              |
|                  |            | 0x0004    | NMI                     |
|                  |            | 0x0000    | Reset                   |
|                  |            |           | Initial SP value        |

Figure 2-2 Vector table

## 3.5 - EXCEPTIONS - SERVICING

---

### 7- Exception Servicing

The handler must service the interrupt request, in this process at least three actions must be taken:

- 1- The interrupt request signal must be deactivated, otherwise the processor would continuously be servicing this interrupt.
- 2- Any volatile data (such as a byte that arrived on the UART and is available in the Receiving Register) must be saved.
- 3- Apart from the registers saved in Step 4 of the entry process, any other register must be saved by the handler before modifying it and these registers must be restored before returning to the interrupted code.

## 3.5 - EXCEPTIONS - RETURN

---

### 8a- Exception Return

The instruction that causes the return from the handler to the interrupted code is either:

`BX LR`      or

`POP { . . . , PC }`      // if this instruction is used the the entry of the handler must be `PUSH { . . . , LR }`

## 3.5 - EXCEPTIONS - RETURN

### 8b - Exception Return

What happens when a value such as 0xFFFF FFF1 is loaded to the PC ?  
Being an invalid code address, the processor detects that this is an EXC\_RETURN code and proceeds with the actions described in the table in the [slide](#) Step 5 of the Interrupt Entry.

| EXC_RETURN[31:0] | Description                                                                                                              |
|------------------|--------------------------------------------------------------------------------------------------------------------------|
| 0xFFFFFFF1       | Return to Handler mode, exception return uses non-floating-point state from the MSP and execution uses MSP after return. |
| 0xFFFFFFF9       | Return to Thread mode, exception return uses non-floating-point state from MSP and execution uses MSP after return.      |
| 0xFFFFFFFD       | Return to Thread mode, exception return uses non-floating-point state from the PSP and execution uses PSP after return.  |
| 0xFFFFFFE1       | Return to Handler mode, exception return uses floating-point-state from MSP and execution uses MSP after return.         |
| 0xFFFFFE9        | Return to Thread mode, exception return uses floating-point state from MSP and execution uses MSP after return.          |
| 0xFFFFFED        | Return to Thread mode, exception return uses floating-point state from PSP and execution uses PSP after return.          |

# INTERRUPT CONTROLLER

---

- **NVIC**
  - Structure
  - Registers
- **CMSIS interface for NVIC operations**

# NVIC

---

The Nested Vectored Interrupt Controller (NVIC) is an integral part of the Cortex-M4 architecture. Hence, all Cortex-M4 have the same interrupt controller.

Exception handling was examined in Section 3.5 ([link](#))

Functionality of the NVIC:

- **detect** interrupt requests (IRQ) at its inputs and **combine** them into a single interrupt request to the microprocessor core
- capability to **mask** any given input
- associate inputs to interrupt **priority** levels

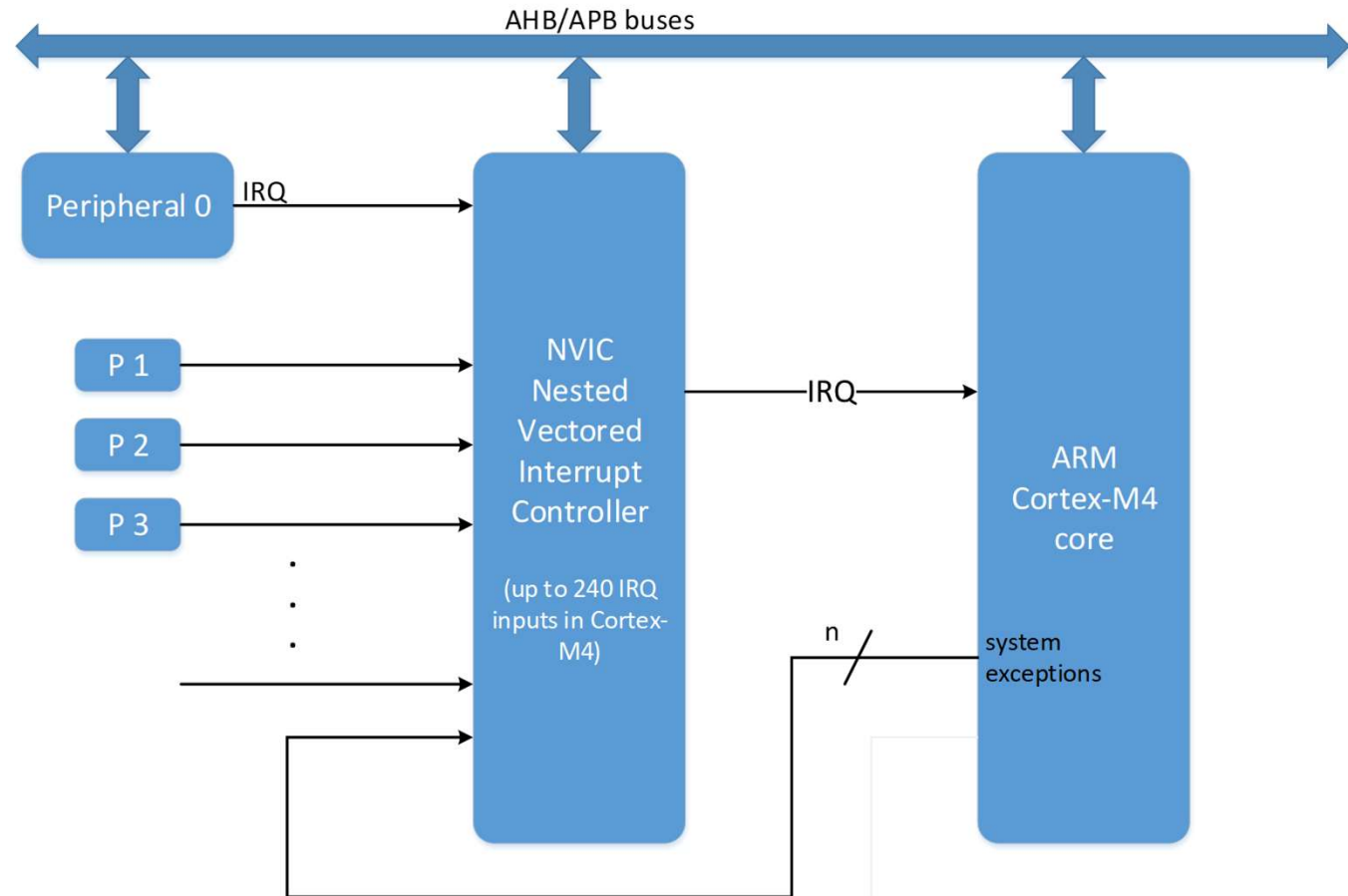
# NVIC

The Cortex-M4 NVIC may have up to 240 IRQ inputs.

Each input is an interrupt request coming from an integrated peripheral, an external pin, or even from the core itself to inform of a system exception.

The operation of the NVIC is configured by the core via a set of NVIC registers that provide the functionality of:

- input masking
- IRQ priority assignment
- check status of input (is there a pending IRQ?)



# NVIC CHARACTERISTICS

---

The Nested Vectored Interrupt Controller provides support for:

- **Nested exceptions** – each exception has an associated priority level. Nesting means that when an exception is being serviced, it can be interrupted by a higher priority exception that was activated. Once the higher priority exception has its service completed, the lower priority service resumes.
- **Vectored exceptions** – the starting addresses of the service routines (handlers) are stored in a table (vector table). When IRQ<sub>i</sub> is detected the processor simply reads entry *i* of the table and starts servicing, avoiding delays to start the handler.
- **Interrupt masking** – each IRQ<sub>i</sub> can be individually masked, i.e. ignored.

## VECTOR TABLE

---

By default, its start address is 0x0.

The table has a 31-bit address for each of the possible exceptions (number is implementation dependent).

The 32th bit (the LSb) is used to identify the instruction set (ARM or Thumb) and must be always set in a Cortex-M processor

The first entry in the table is the initial value of the SP.

The next 15 entries are for the system exceptions (Reset, NMI, ... SysTick).

Then follow up to 240 entries for IRQ0 to IRQ239.

# VECTOR TABLE FOR THE RENESAS S7G2

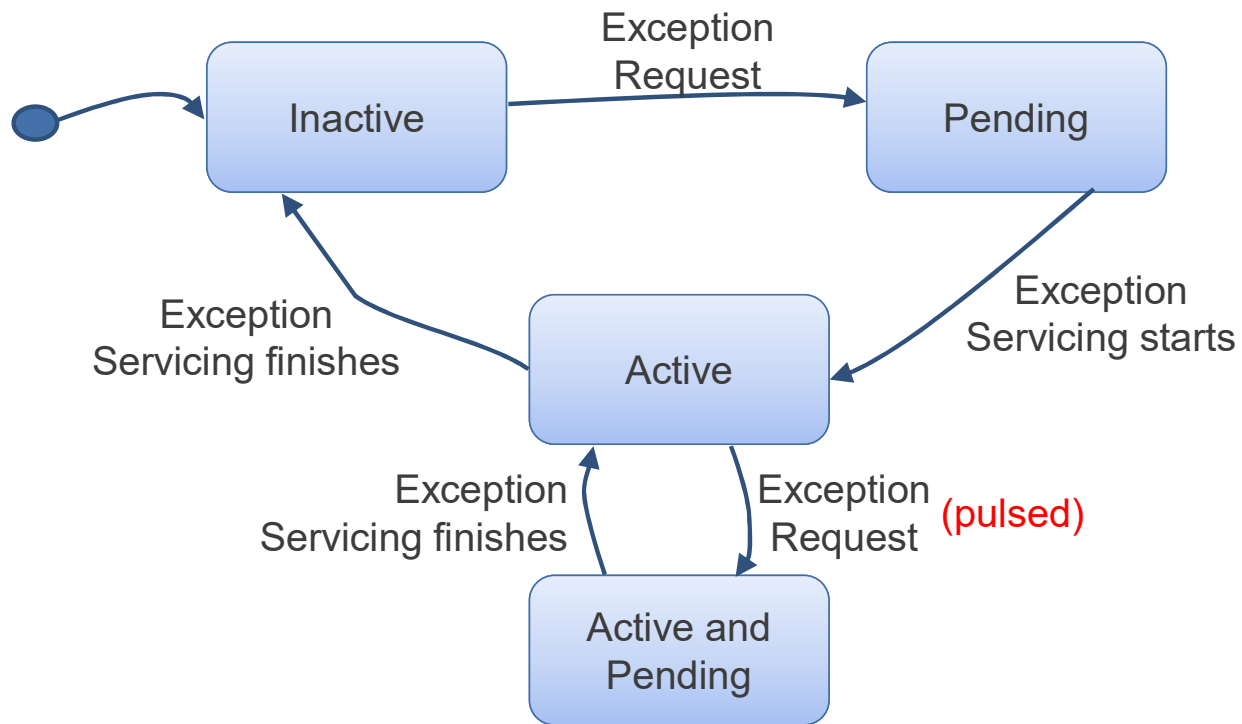
On the S7G2 the interrupt sources are managed by the ICU – Interrupt Control Unit.

The ICU sits between the peripherals and the NVIC, preprocessing the interrupt requests before sending them to the NVIC.

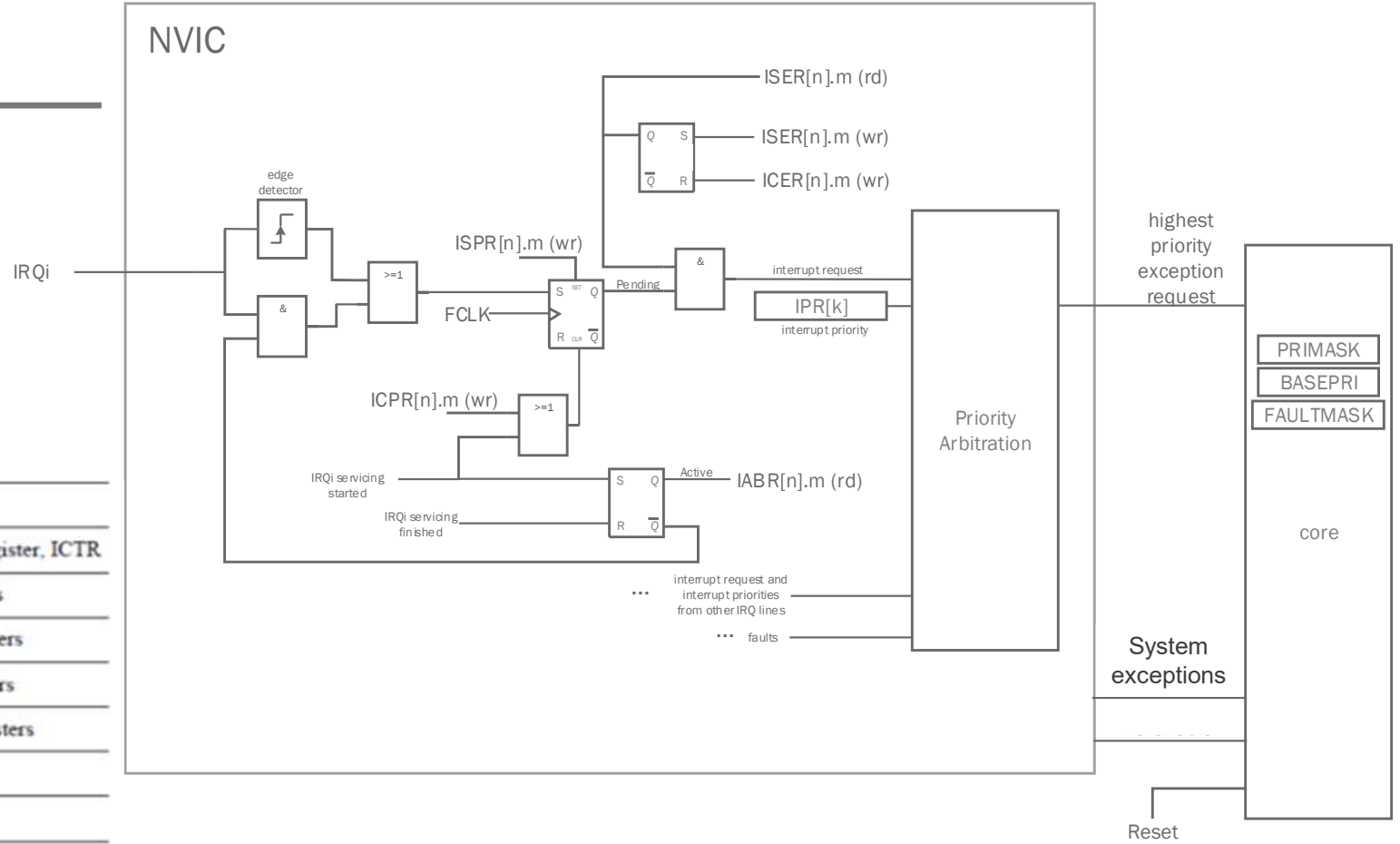
**Table 14.3** Interrupt vector table (1/3)

| Exception number | IRQ number | Vector offset | Source      | Description                                          |
|------------------|------------|---------------|-------------|------------------------------------------------------|
| 0                | —          | 000h          | ARM         | Initial stack pointer                                |
| 1                | —          | 004h          | ARM         | Initial program counter (Reset Vector)               |
| 2                | —          | 008h          | ARM         | Non-maskable interrupt (NMI)                         |
| 3                | —          | 00Ch          | ARM         | Hard fault                                           |
| 4                | —          | 010h          | ARM         | MemManage fault                                      |
| 5                | —          | 014h          | ARM         | Bus fault                                            |
| 6                | —          | 018h          | ARM         | Usage fault                                          |
| 7                | —          | 01Ch          | ARM         | Reserved                                             |
| 8                | —          | 020h          | ARM         | Reserved                                             |
| 9                | —          | 024h          | ARM         | Reserved                                             |
| 10               | —          | 028h          | ARM         | Reserved                                             |
| 11               | —          | 02Ch          | ARM         | Supervisor call (SVCall)                             |
| 12               | —          | 030h          | ARM         | Debug Monitor                                        |
| 13               | —          | 034h          | ARM         | Reserved                                             |
| 14               | —          | 038h          | ARM         | Pendable request for system service (PendableSrvReq) |
| 15               | —          | 03Ch          | ARM         | System tick timer (SysTick)                          |
| 16               | 0          | 040h          | ICU.IELSR0  | Event selected in the ICU.IELSR0 register            |
| 17               | 1          | 044h          | ICU.IELSR1  | Event selected in the ICU.IELSR1 register            |
| 18               | 2          | 048h          | ICU.IELSR2  | Event selected in the ICU.IELSR2 register            |
| 19               | 3          | 04Ch          | ICU.IELSR3  | Event selected in the ICU.IELSR3 register            |
| ■ ■ ■            |            |               |             |                                                      |
| 107              | 91         | 1ACh          | ICU.IELSR91 | Event selected in the ICU.IELSR91 register           |
| 108              | 92         | 1B0h          | ICU.IELSR92 | Event selected in the ICU.IELSR92 register           |
| 109              | 93         | 1B4h          | ICU.IELSR93 | Event selected in the ICU.IELSR93 register           |
| 110              | 94         | 1B8h          | ICU.IELSR94 | Event selected in the ICU.IELSR94 register           |
| 111              | 95         | 1BCh          | ICU.IELSR95 | Event selected in the ICU.IELSR95 register           |

# EXCEPTION HANDLING STATE MACHINE



## FUNCTIONAL MODEL OF THE NVIC



| Name      | Description                              |
|-----------|------------------------------------------|
| ICTR      | Interrupt Controller Type Register, ICTR |
| NVIC_ISER | Interrupt Set-Enable Registers           |
| NVIC_ICER | Interrupt Clear-Enable Registers         |
| NVIC_ISPR | Interrupt Set-Pending Registers          |
| NVIC_ICPR | Interrupt Clear-Pending Registers        |
| NVIC_IABR | Interrupt Active Bit Register            |
| NVIC_IPR0 | Interrupt Priority Register              |

## NVIC - ARM DOCUMENTATION

---

- ARMv7-M Architecture Reference Manual (2014)  
[ARM DDI 0403E.b](#)
- ARM® Cortex® -M4 Processor Revision: r0p1 (2015)  
Technical Reference Manual  
[ARM 100166 0001 00 en](#)
- Cortex™-M4 Devices Generic User Guide (2011)  
[ARM DUI 0553A](#)

# NVIC REGISTERS

---

**Table 6-1 NVIC registers**

| Address                 | Name                    | Type | Reset      | Description                              |
|-------------------------|-------------------------|------|------------|------------------------------------------|
| 0xE000E004              | ICTR                    | RO   | -          | Interrupt Controller Type Register, ICTR |
| 0xE000E100 - 0xE000E11C | NVIC_ISER0 - NVIC_ISER7 | RW   | 0x00000000 | Interrupt Set-Enable Registers           |
| 0xE000E180 - 0xE000E19C | NVIC_ICER0 - NVIC_ICER7 | RW   | 0x00000000 | Interrupt Clear-Enable Registers         |
| 0xE000E200 - 0xE000E21C | NVIC_ISPR0 - NVIC_ISPR7 | RW   | 0x00000000 | Interrupt Set-Pending Registers          |
| 0xE000E280 - 0xE000E29C | NVIC_ICPR0 - NVIC_ICPR7 | RW   | 0x00000000 | Interrupt Clear-Pending Registers        |
| 0xE000E300 - 0xE000E31C | NVIC_IABR0 - NVIC_IABR7 | RO   | 0x00000000 | Interrupt Active Bit Register            |
| 0xE000E400 - 0xE000E4EC | NVIC_IPR0 - NVIC_IPR59  | RW   | 0x00000000 | Interrupt Priority Register              |

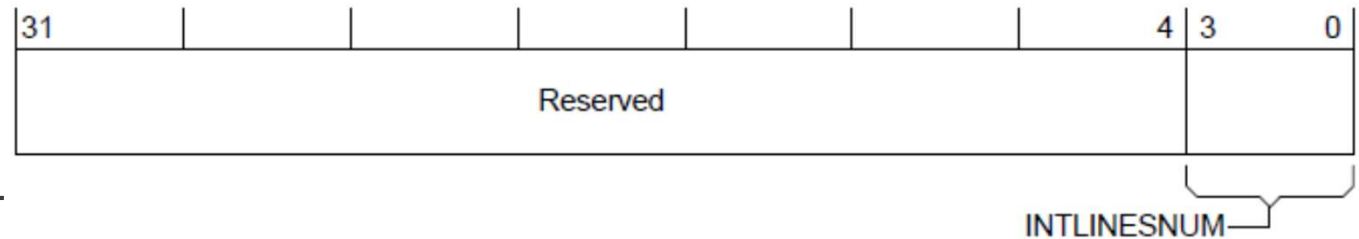
source: ARM® Cortex® -M4 Technical Reference [Manual](#)

## NVIC REGISTERS - ICTR

Since the number of input lines (IRQi) to the NVIC is implementation dependent, so is the number of registers to control the NVIC.

ICTR informs how many control registers are actually implemented in any given NVIC.

For the Renesas S7G2, ICTR.INTLINESNUM holds the value 2, meaning that there are 3 registers of each type (ISER, ICER, ISPR, ICPR, IABR, with indexes 0..2; and 24 registers IPR, with indexes 0..23). The number of input lines IRQi on this processor is limited to 96.



**Figure 6-1 ICTR bit assignments**

[3:0] INTLINESNUM Total number of interrupt lines in groups of 32:

0b0000 = 0...32

0b0001 = 33...64

0b0010 = 65...96

0b0011 = 97...128

0b0100 = 129...160

0b0101 = 161...192

0b0110 = 193...224

0b0111 = 225...256

## NVIC REGISTERS – ISER – S7G2

---

|         | bit31 |       |     | bit0  |       |       |
|---------|-------|-------|-----|-------|-------|-------|
| ISER[0] | IRQ31 | IRQ30 | ... | IRQ2  | IRQ1  | IRQ0  |
| ISER[1] | IRQ63 | IRQ62 | ... | IRQ34 | IRQ33 | IRQ32 |
| ISER[2] | IRQ95 | IRQ94 | ... | IRQ66 | IRQ65 | IRQ64 |

In a Renesas S7G2, each bit of registers ISER[0] to ISER[2] corresponds to one of the 96 IRQ<sub>i</sub> lines.

on write: 1 enables the interrupt line, 0 has no effect

on read: returns the current state of each IRQ<sub>i</sub> line

0 = disabled, 1 = enabled

## NVIC REGISTERS – ICER/ISPR/ICPR/IABR – S7G2

---

Same bit assignment as for ISER.

ICER: on write: 1 **disables** the interrupt line, 0 has no effect

on read: report the current state of each IRQi line, 0 = disabled, 1 = enabled

ISPR: on write: 1 sets the interrupt line to **pending**, 0 has no effect

on read: report the current state of each IRQi line, 0 = not-pending, 1 = pending

ICPR: on write: 1 clears the pending state of the interrupt line, 0 has no effect

on read: report the current state of each IRQi line, 0 = not-pending, 1 = pending

IABR: read only: report the current state of each IRQi line, 0 = not-active, 1 = active

## NVIC PRIORITIES

---

On the Cortex-M4, each IRQi line has a register that defines its priority level in relation to the other IRQi lines. This register may have up to 8 bits (implementation defined).

On the S7G2, 4-bit registers are implemented, allowing the definition of up to 16 priority levels. 0 is the highest priority and 15 is the lowest. Since these bits are left aligned, the actual values are 0, 0x10, 0x20, 0x30, ... 0xF0.

**A higher priority interrupt preempts a lower priority interrupt whose handler is being executed.**

## NVIC PRIORITIES

---

The bits of the priority register are divided into:

- Priority Group
- Priority Subgroup

The bits in the Priority Group define the number of priority levels for the purpose of preemption.

The bits in the Priority Subgroup define the number of sub-levels for the purpose of selecting which IRQ<sub>i</sub> will be serviced first if two IRQ<sub>i</sub> are simultaneously pending when the core accepts an interrupt.

## NVIC PRIORITIES

---

The PRIGROUP bits of the AIRCR register are used to configure the division of bits among Priority Group and Subgroup.

AIRCR.PRIGROUP may be written with values in the range of 0 .. 7

On the S7G2, programming PRIGROUP with 0, 1, 2 or 3 has the effect of using the four bits for Priority Group and none for the Subgroup.

For PRIGROUP = 4 the division is 3.1 (3 for Group and 1 for Subgroup)

For PRIGROUP = 5 the division is 2.2, and so on...

# CMSIS-CORE INTERRUPT FUNCTIONS

These is a partial list of the functions available in CMSIS-CORE that provide access to the NVIC as well as to other interrupt functionality.

Next slides detail these functions

| CMSIS function                                                         | Description                                    |
|------------------------------------------------------------------------|------------------------------------------------|
| <code>void NVIC_SetPriorityGrouping (uint32_t PriorityGroup)</code>    | Set priority grouping                          |
| <code>uint32_t NVIC_GetPriorityGrouping (void)</code>                  | Read the priority grouping                     |
| <code>void NVIC_EnableIRQ (IRQn_Type IRQn)</code>                      | Enable a device-specific interrupt             |
| <code>uint32_t NVIC_GetEnableIRQ (IRQn_Type IRQn)</code>               | Get a device-specific interrupt enable status. |
| <code>void NVIC_DisableIRQ (IRQn_Type IRQn)</code>                     | Disable a device-specific interrupt            |
| <code>uint32_t NVIC_GetPendingIRQ (IRQn_Type IRQn)</code>              | Get the pending device-specific interrupt      |
| <code>void NVIC_SetPendingIRQ (IRQn_Type IRQn)</code>                  | Set a device-specific interrupt to pending     |
| <code>void NVIC_ClearPendingIRQ (IRQn_Type IRQn)</code>                | Clear a device-specific interrupt from pending |
| <code>uint32_t NVIC_GetActive (IRQn_Type IRQn)</code>                  | Get the device-specific interrupt active       |
| <code>void NVIC_SetPriority (IRQn_Type IRQn, uint32_t priority)</code> | Set the priority for an interrupt              |
| <code>uint32_t NVIC_GetPriority (IRQn_Type IRQn)</code>                | Get the priority of an interrupt               |

# CMSIS-CORE INTERRUPT FUNCTIONS

---

```
typedef enum {  
/* ----- Cortex-M4 Processor Exceptions Numbers ----- */  
Reset_IRQn           = -15,      /*!< 1  Reset Vector, invoked on Power up and warm reset           */  
NonMaskableInt_IRQn = -14,      /*!< 2  Non maskable Interrupt, cannot be stopped or preempted      */  
HardFault_IRQn       = -13,      /*!< 3  Hard Fault, all classes of Fault                             */  
MemoryManagement_IRQn = -12,    /*!< 4  Memory Management, MPU mismatch, including Access Violation  
and No Match                                                    */  
BusFault_IRQn        = -11,      /*!< 5  Bus Fault, Pre-Fetch-, Memory Access Fault, other address/memory  
related Fault                                                    */  
UsageFault_IRQn       = -10,      /*!< 6  Usage Fault, i.e. Undef Instruction, Illegal State Transition */  
SVCall_IRQn           = -5,      /*!< 11 System Service Call via SVC instruction                      */  
DebugMonitor_IRQn     = -4,      /*!< 12 Debug Monitor                                                  */  
PendSV_IRQn           = -2,      /*!< 14 Pendable request for system service                          */  
SysTick_IRQn          = -1,      /*!< 15 System Tick Timer                                             */  
} IRQn_Type;
```

This is the IRQn\_Type enumeration defined in file S7G2.h

# CMSIS-CORE INTERRUPT FUNCTIONS

```
void __enable_irq(void)
```

Resets PRIMASK in the core, allowing interrupts from NVIC to be serviced.

Example: `__enable_irq( );`

```
void __disable_irq(void)
```

Sets PRIMASK in the core, preventing exceptions with configurable priorities (exceptions 4 and up) to be serviced.

Example: `__disable_irq( );`

|  |           |          |  |  |  |  |  |   |   |  |  |         |    |
|--|-----------|----------|--|--|--|--|--|---|---|--|--|---------|----|
|  |           | 31       |  |  |  |  |  | 8 | 7 |  |  | 1       | 0  |
|  | PRIMASK   | Reserved |  |  |  |  |  |   |   |  |  |         | PM |
|  | FAULTMASK | Reserved |  |  |  |  |  |   |   |  |  |         | FM |
|  | BASEPRI   | Reserved |  |  |  |  |  |   |   |  |  | BASEPRI |    |

| Field   | bit                          | Description                                                                                                                                                                                                                        |
|---------|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PM      | <a href="#">PRIMASK[0]</a>   | Set to mask exceptions with configurable priority (priority 0 and lower). Reset to unmask. When PRIMASK is set, only Reset, NMI and <a href="#">HardFault</a> are accepted.                                                        |
| FM      | <a href="#">FAULTMASK[0]</a> | Set to mask the <a href="#">HardFault</a> exceptions and the configurable priorities (prio -1 and lower). When FAULTMASK is set, only Reset and NMI are accepted, i.e. <a href="#">HardFault</a> is masked.                        |
| BASEPRI | <a href="#">BASEPRI[7:0]</a> | Changes the priority level required for exception preemption. Affects only the currently executing code with lower priority than BASEPRI. BASEPRI = 0 disables it. BASEPRI = n indicates that priority levels n and up are masked. |

# CMSIS-CORE INTERRUPT FUNCTIONS

```
void NVIC_EnableIRQ(IRQn_Type)
```

Enables (**unmask**) a specific IRQi line

Example:

```
NVIC_Enable(SysTick_IRQn);
```

```
void NVIC_DisableIRQ(IRQn_Type)
```

Disables (**mask**) a specific IRQi line

Example:

```
NVIC_Disable(SysTick_IRQn);
```

## NVIC REGISTERS – ISER – S7G2

|         | bit31 |       |     |     | bit0  |       |       |
|---------|-------|-------|-----|-----|-------|-------|-------|
| ISER[0] | IRQ31 | IRQ30 | ... | ... | IRQ2  | IRQ1  | IRQ0  |
| ISER[1] | IRQ63 | IRQ62 | ... | ... | IRQ34 | IRQ33 | IRQ32 |
| ISER[2] | IRQ95 | IRQ94 | ... | ... | IRQ66 | IRQ65 | IRQ64 |

In a Renesas S7G2, each bit of registers ISER[0] to ISER[2] corresponds to one of the 96 IRQi lines.

on write: 1 enables the interrupt line, 0 has no effect

on read: returns the current state of each IRQi line

0 = disabled, 1 = enabled

# CMSIS-CORE INTERRUPT FUNCTIONS

```
uint32_t NVIC_GetPendingIRQ(IRQn_Type)
```

Reads the Pending status (P\_FF) returning 0 (not pending) or 1 (pending)

**Example:** `uint32_t pend = NVIC_GetPendingIRQ(SysTick_IRQn);`

```
void NVIC_SetPendingIRQ(IRQn_Type)
```

Sets the Pending status (P-FF) of a specific IRQi line

**Example:** `NVIC_SetPendingIRQ(SysTick_IRQn);`

```
void NVIC_ClearPendingIRQ(IRQn_Type)
```

Clears (resets) the Pending status (P-FF) of a specific IRQi line

**Example:** `NVIC_ClearPendingIRQ(SysTick_IRQn);`

## NVIC REGISTERS – ICER/ISPR/ICPR/IABR – S7G2

Same bit assignment as for ISER.

ICER: on write: 1 **disables** the interrupt line, 0 has no effect  
on read: report the current state of each IRQi line, 0 = disabled, 1 = enabled

ISPR: on write: 1 sets the interrupt line to **pending**, 0 has no effect  
on read: report the current state of each IRQi line, 0 = not-pending, 1 = pending

ICPR: on write: 1 clears the pending state of the interrupt line, 0 has no effect  
on read: report the current state of each IRQi line, 0 = not-pending, 1 = pending

IABR: read only: report the current state of each IRQi line, 0 = not-active, 1 = active

## CMSIS-CORE INTERRUPT FUNCTIONS

---

```
void NVIC_SetPriority(IRQn_Type IRQn, uint32_t priority)
```

Sets the priority of a specific IRQi line

**Example:** `NVIC_SetPriority(SysTick_IRQn, 4);`

```
uint32_t NVIC_GetPriority(IRQn_Type IRQn)
```

Get the priority level of a specific IRQi line

**Example:** `uint32_t prio = NVIC_GetPriority(SysTick_IRQn);`

Rem: The integer value of the priority must be in the range of  $0 \dots 2^N - 1$ , where N is the number of priority bits implemented. For S7G2 the range is 0..15.

---

# CMSIS-CORE INTERRUPT FUNCTIONS

---

```
uint32_t NVIC_GetActive (IRQn_Type)
```

Reads the Active status (ACTIVE\_FF) returning 0 (not active) or 1 (active)

Example: `uint32_t act = NVIC_GetActive (SysTick_IRQn) ;`



| NVIC REGISTERS – ICER/ISPR/ICPR/IABR – S7G2 |                                                                                                                                                                         |
|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Same bit assignment as for <u>ISER</u> .    |                                                                                                                                                                         |
| <u>ICER</u> :                               | on write: 1 <b>disables</b> the interrupt line, 0 has no effect<br>on read: report the current state of each <u>IRQi</u> line, 0 = disabled, 1 = enabled                |
| <u>ISPR</u> :                               | on write: 1 sets the interrupt line to <b>pending</b> , 0 has no effect<br>on read: report the current state of each <u>IRQi</u> line, 0 = not-pending, 1 = pending     |
| <u>ICPR</u> :                               | on write: 1 clears the pending state of the interrupt line, 0 has no effect<br>on read: report the current state of each <u>IRQi</u> line, 0 = not-pending, 1 = pending |
| <u>IABR</u> :                               | read only: report the current state of each <u>IRQi</u> line, 0 = not-active, 1 = active                                                                                |

[illegible]

Older documentation describes this field as the vector table address offset. The description in this manual clarifies the use of this register.

| Address    | Name  | Type | Reset                     | Description                                                                                                                                                                                                                                                                                    |
|------------|-------|------|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0xE000ED00 | CPUID | RO   | IMPLEMENTATION<br>DEFINED | <i>CPUID Base Register</i> on page B3-598.                                                                                                                                                                                                                                                     |
| 0xE000ED04 | ICSR  | RW   | 0x00000000                | <i>Interrupt Control and State Register; ICSR</i> on page B3-599.                                                                                                                                                                                                                              |
| 0xE000ED08 | VTOR  | RW   | 0x00000000 <sup>a</sup>   | <i>Vector Table Offset Register; VTOR</i> on page B3-601.                                                                                                                                                                                                                                      |
| 0xE000ED0C | AIRCR | RW   | b                         | <i>Application Interrupt and Reset Control Register; AIRCR</i> on page B3-601.                                                                                                                                                                                                                 |
| 0xE000ED10 | SCR   | RW   | 0x00000000                | <i>System Control Register; SCR</i> on page B3-603.                                                                                                                                                                                                                                            |
| 0xE000ED14 | CCR   | RW   | IMPLEMENTATION<br>DEFINED | <i>Configuration and Control Register; CCR</i> on page B3-604.                                                                                                                                                                                                                                 |
| Reserved   |       |      | 0x00000000                | <i>System Handler Priority Register 1, SHPR1</i> on page B3-606.                                                                                                                                                                                                                               |
|            |       |      | 0x00000000                | <i>System Handler Priority Register 2, SHPR2</i> on page B3-606.                                                                                                                                                                                                                               |
|            |       |      | 0x00000000                | <i>System Handler Priority Register 3, SHPR3</i> on page B3-607.                                                                                                                                                                                                                               |
|            |       |      | 0x00000000                | <i>System Handler Control and State Register; SHCSR</i> on page B3-607.                                                                                                                                                                                                                        |
|            |       |      | 0x00000000                | <i>Configurable Fault Status Register; CFSR</i> on page B3-609.                                                                                                                                                                                                                                |
|            |       |      |                           | The following describe the subregisters of the CFSR: <ul style="list-style-type: none"> <li><i>MemManage Status Register; MMFSR</i> on page B3-609.</li> <li><i>BusFault Status Register; BFSR</i> on page B3-610.</li> <li><i>UsageFault Status Register; UFSR</i> on page B3-611.</li> </ul> |

# INTERRUPT SOURCES

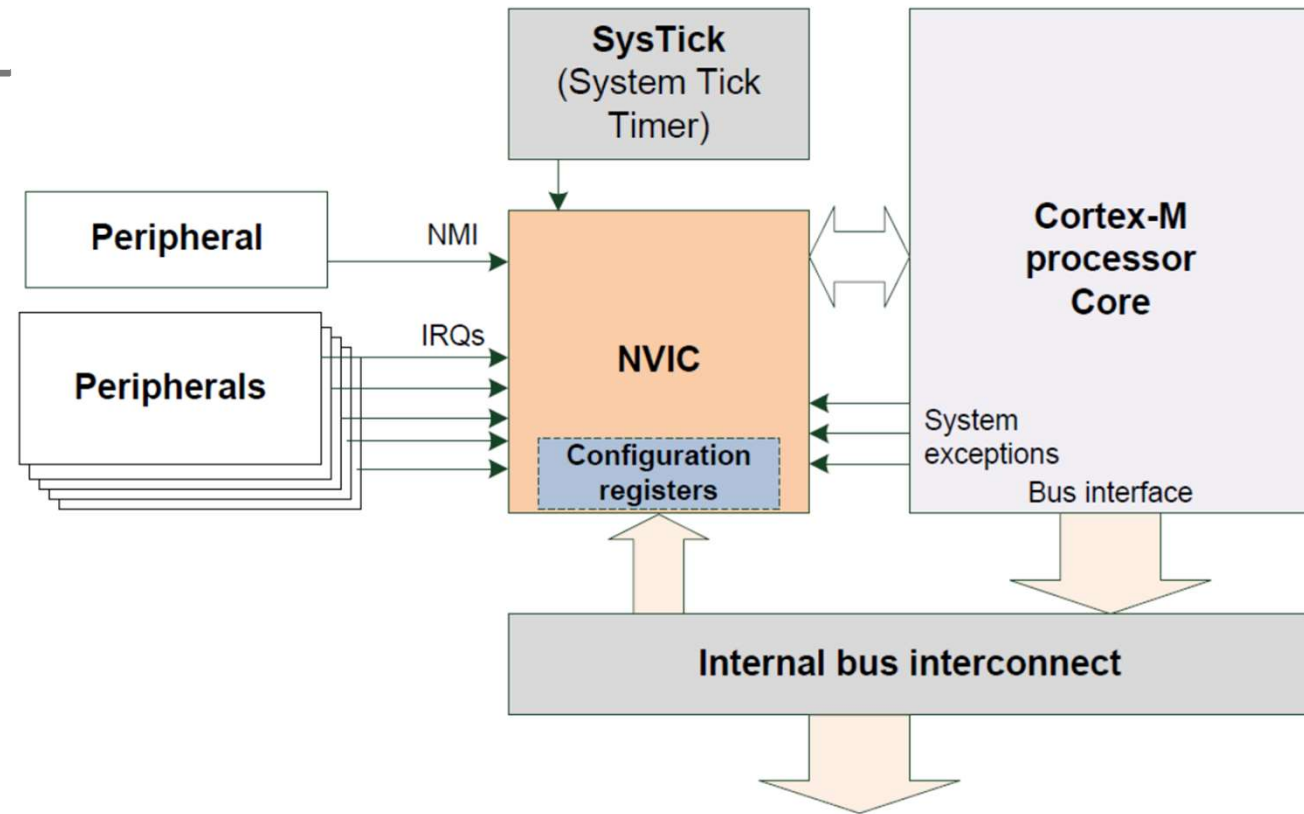
---

Possible **interrupt** sources are:

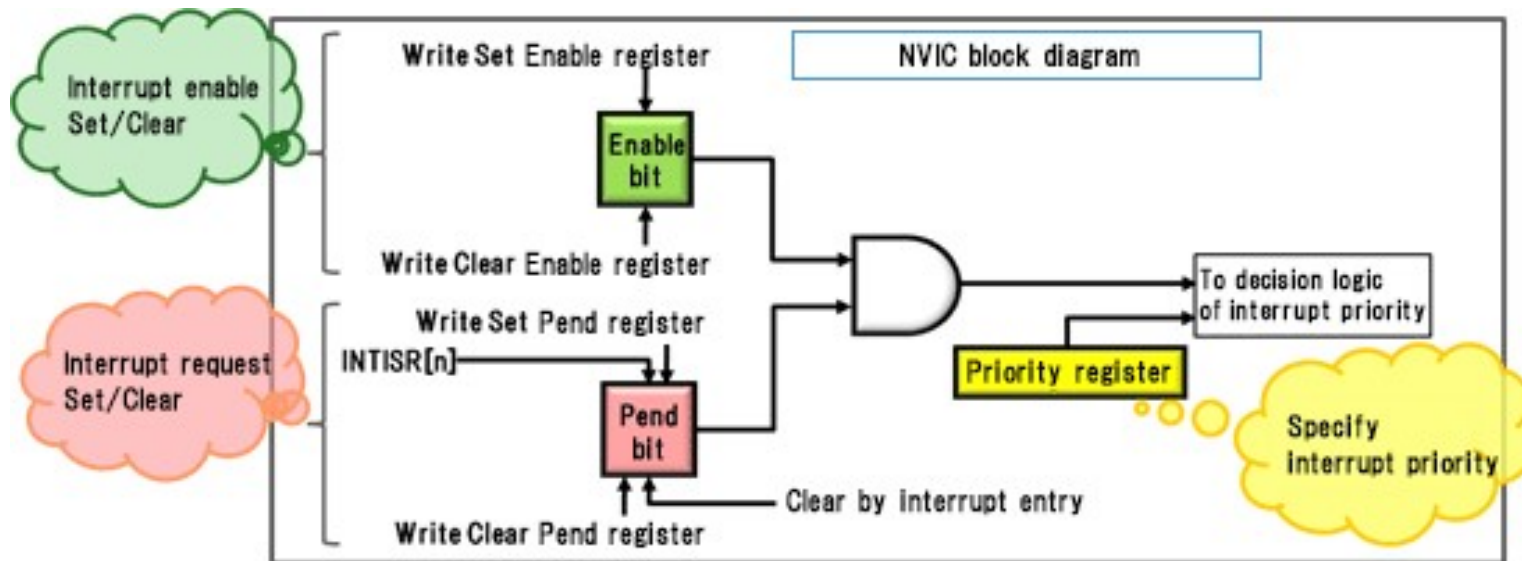
- up to 240 IRQ lines (HW)
- NMI (e.g. brown-out detection)
- SysTick
- *Rem: NVIC also manages the System exceptions*

# NVIC

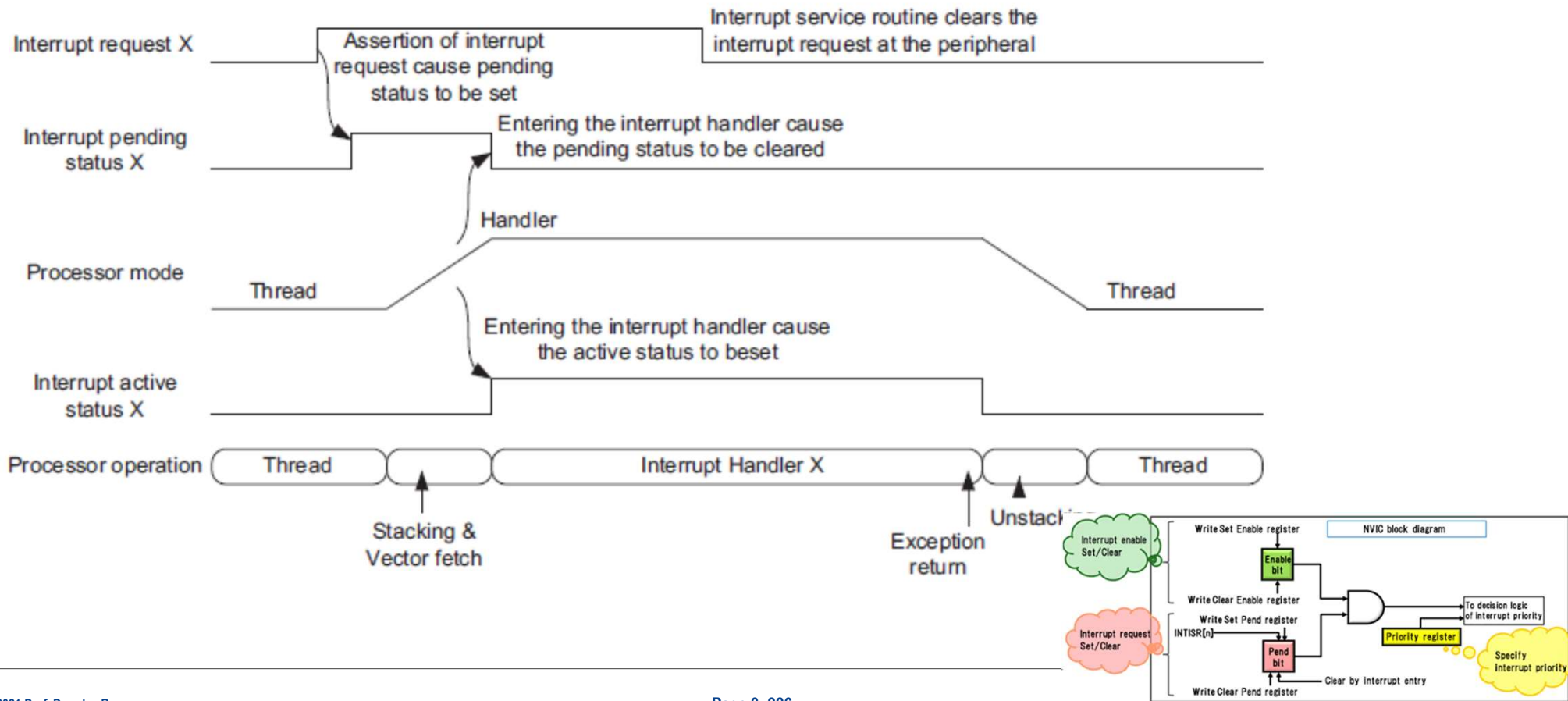
---



# NVIC OPERATION



Cortex-M4 accepts both level and edge-sensitive irqs, no need to configure. Pending status is cleared **automatically** when handler is started.



# Cortex Microcontroller Software Interface Standard (CMSIS)

## CMSIS

CMSIS functions enable software portability between different Cortex-M profile processors. To access the NVIC registers when using CMSIS, use the following functions:

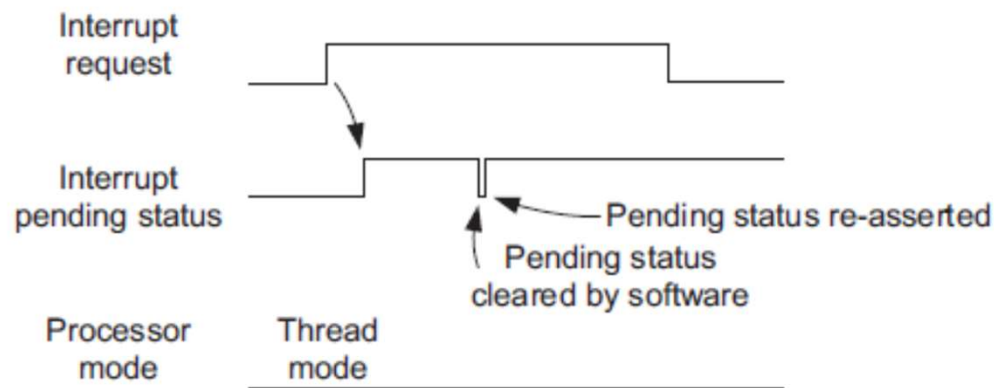
**Table 4-3 CMSIS access NVIC functions**

| <b>CMSIS function</b>                                                             | <b>Description</b>                                                                                                                 |
|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <code>void NVIC_EnableIRQ(IRQn_Type IRQn)<sup>a</sup></code>                      | Enables an interrupt or exception.                                                                                                 |
| <code>void NVIC_DisableIRQ(IRQn_Type IRQn)<sup>a</sup></code>                     | Disables an interrupt or exception.                                                                                                |
| <code>void NVIC_SetPendingIRQ(IRQn_Type IRQn)<sup>a</sup></code>                  | Sets the pending status of interrupt or exception to 1.                                                                            |
| <code>void NVIC_ClearPendingIRQ(IRQn_Type IRQn)<sup>a</sup></code>                | Clears the pending status of interrupt or exception to 0.                                                                          |
| <code>uint32_t NVIC_GetPendingIRQ(IRQn_Type IRQn)<sup>a</sup></code>              | Reads the pending status of interrupt or exception. This function returns non-zero value if the pending status is set to 1.        |
| <code>void NVIC_SetPriority(IRQn_Type IRQn, uint32_t priority)<sup>a</sup></code> | Sets the priority of an interrupt or exception with configurable priority level to 1.                                              |
| <code>uint32_t NVIC_GetPriority(IRQn_Type IRQn)<sup>a</sup></code>                | Reads the priority of an interrupt or exception with configurable priority level. This function return the current priority level. |

Cortex-M4 accepts both cases, no need to configure.  
Pending status is cleared automatically when handler is started.

## LEVEL VS EDGE INTERRUPTS

---

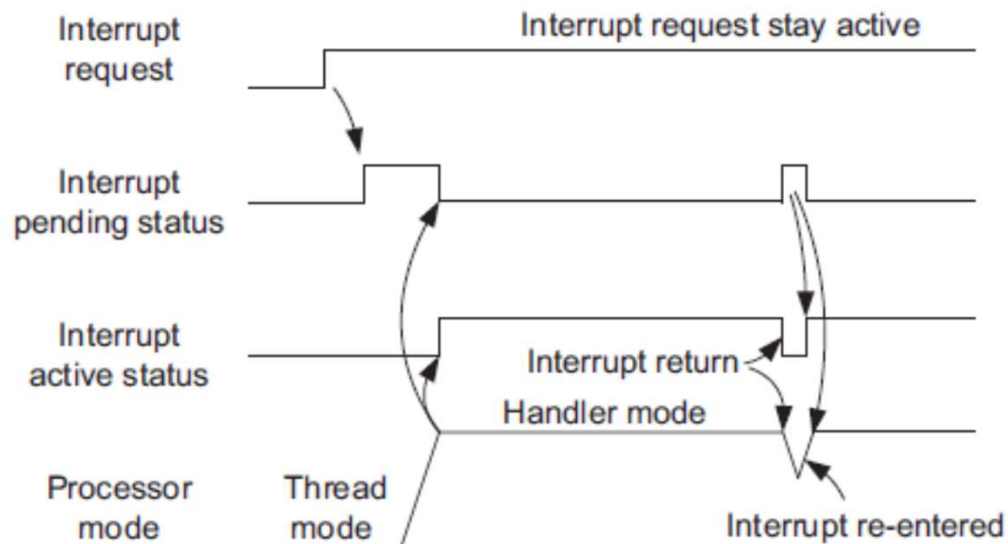


**FIGURE 7.16**

Interrupt pending status cleared and re-asserted due to continuous interrupt request

Cortex-M4 accepts both cases, no need to configure.  
Pending status is cleared automatically when handler is started.

## LEVEL VS EDGE INTERRUPTS

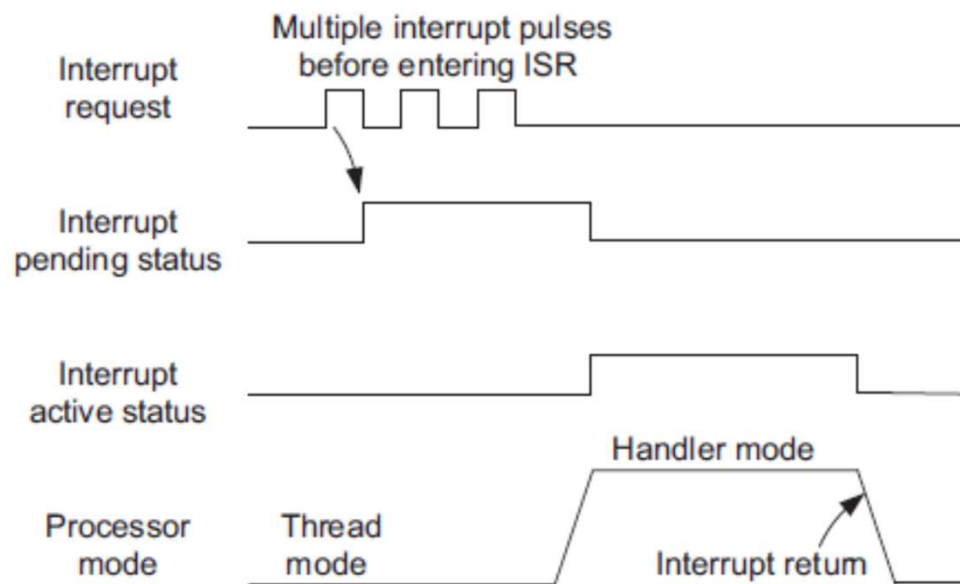


**FIGURE 7.17**

Interrupt gets its pending status set again if the request is still asserted after exception exit

Cortex-M4 accepts both cases, no need to configure.  
Pending status is cleared automatically when handler is started.

## LEVEL VS EDGE INTERRUPTS

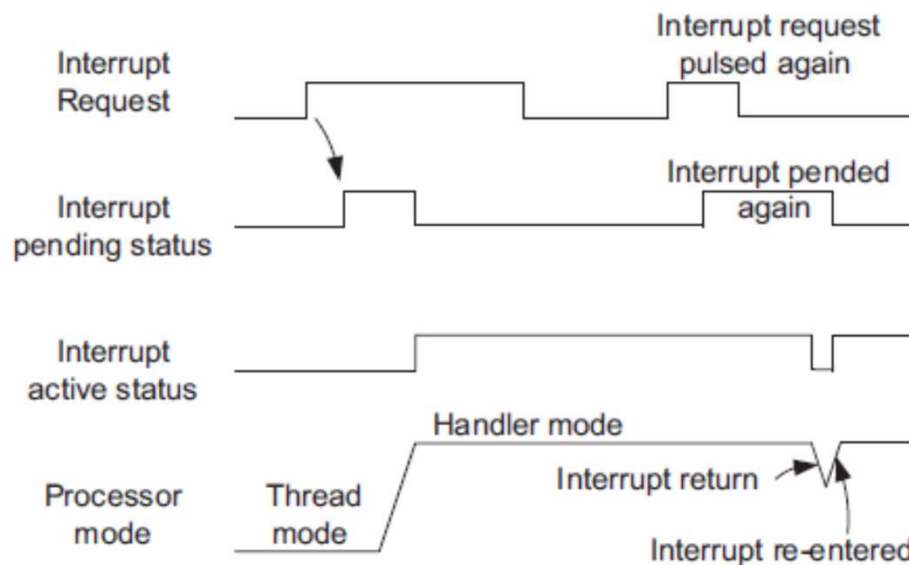


**FIGURE 7.18**

Interrupt pending status set only once with several interrupt request pulses

Cortex-M4 accepts both cases, no need to configure.  
Pending status is cleared automatically when handler is started.

## LEVEL VS EDGE INTERRUPTS



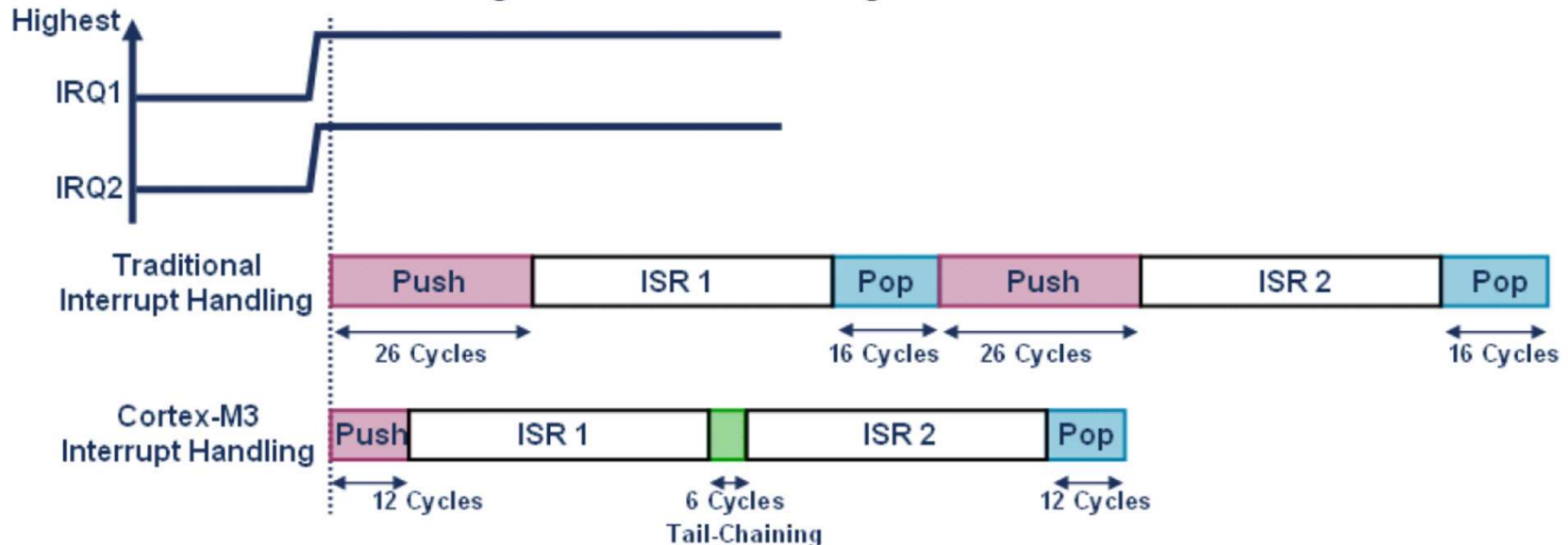
**FIGURE 7.19**

Interrupt pending occurs again during the execution of ISR

## 3.5 - INTERRUPT HANDLING - SPECIAL FEATURES

Tail Chaining: optimization that avoids registers pop followed by registers push when one exception is handled right after another.

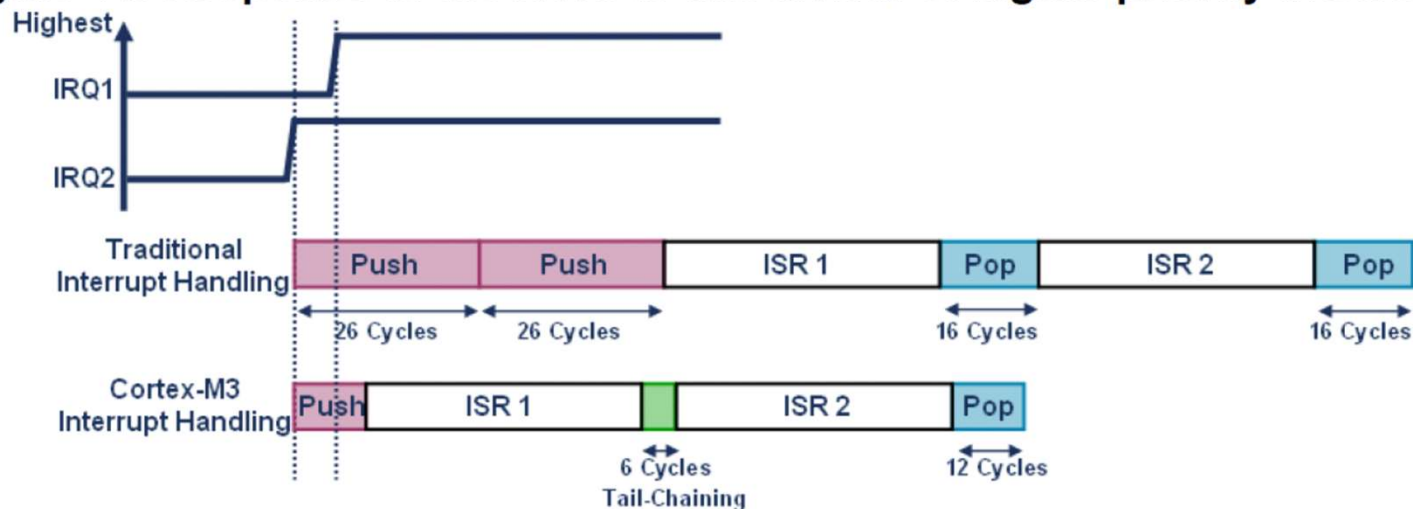
Figure 7. Tail chaining in the NVIC



## 3.5 - INTERRUPT HANDLING - SPECIAL FEATURES

Late arrival: optimization where a higher priority interrupt is serviced first even if it arrives while a prior lower priority interrupt is already in the stage of pushing registers.

**Figure 11. Response of the NVIC to late arrival of higher priority interrupts**



# INTERRUPTS ON THE RENESAS S7G2

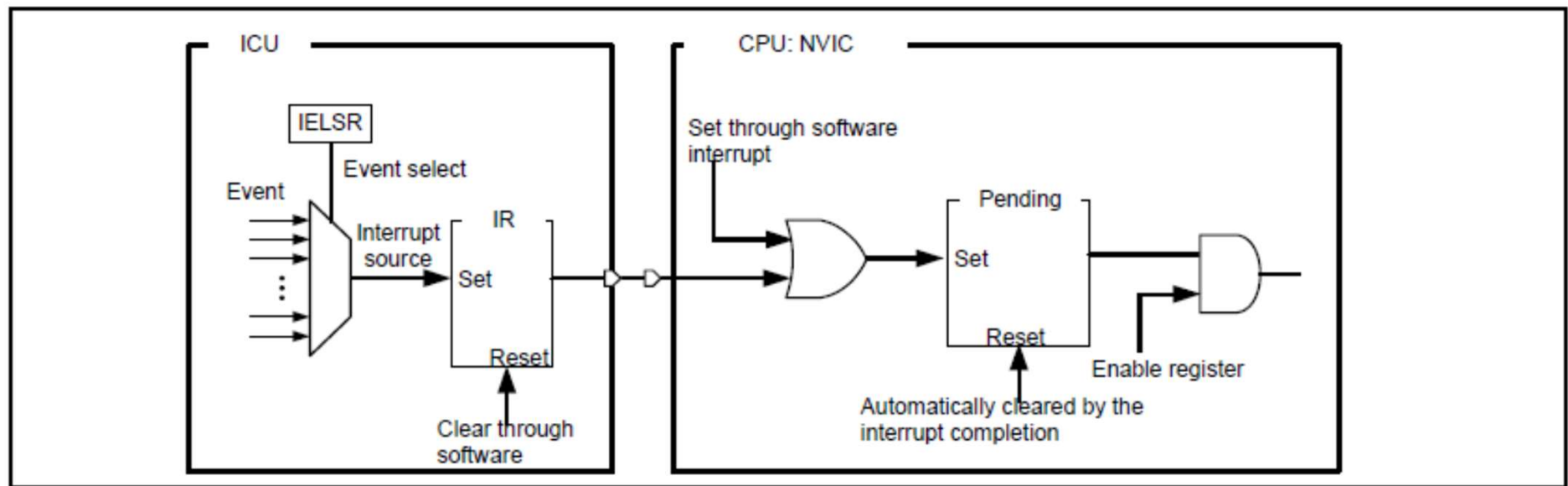


Figure 14.2 Interrupt path of the ICU and CPU: NVIC

## INTERRUPT PRIORITIES

---

ARM defines 8 bits to select the priority level of an exception, hence, 256 levels.

Silicon vendors define the number of bits actually implemented

Renesas defines 4 priority bits for S7G2

These are further divides into group (3 bits) and sub-group (1 bit).

| Bit7                              | Bit6 | Bit5 | Bit4 | Bit3            | Bit2 | Bit1 | Bit0 |
|-----------------------------------|------|------|------|-----------------|------|------|------|
| Implemented<br>16 priority levels |      |      |      | Not implemented |      |      |      |

# INTERRUPT PRIORITIES

---

60 registers (IPR0 to IPR59)

each defines the priority level of 4 IRQ lines



# INTERRUPT PRIORITIES

For system exceptions, the SCB is used to define priorities

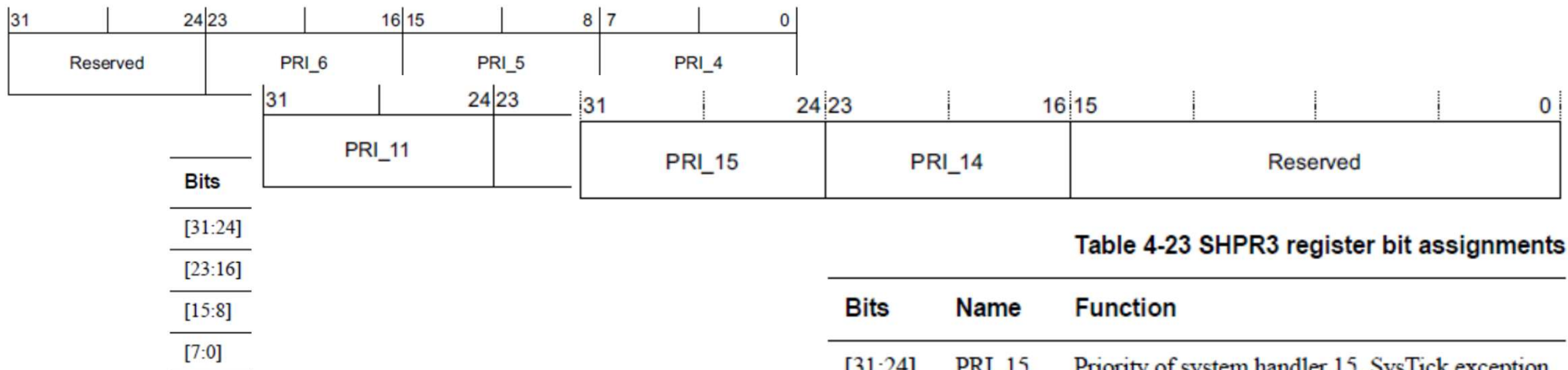


Table 4-23 SHPR3 register bit assignments

| Bits    | Name   | Function                                         |
|---------|--------|--------------------------------------------------|
| [31:24] | PRI_15 | Priority of system handler 15, SysTick exception |
| [23:16] | PRI_14 | Priority of system handler 14, PendSV            |
| [15:0]  | -      | Reserved.                                        |

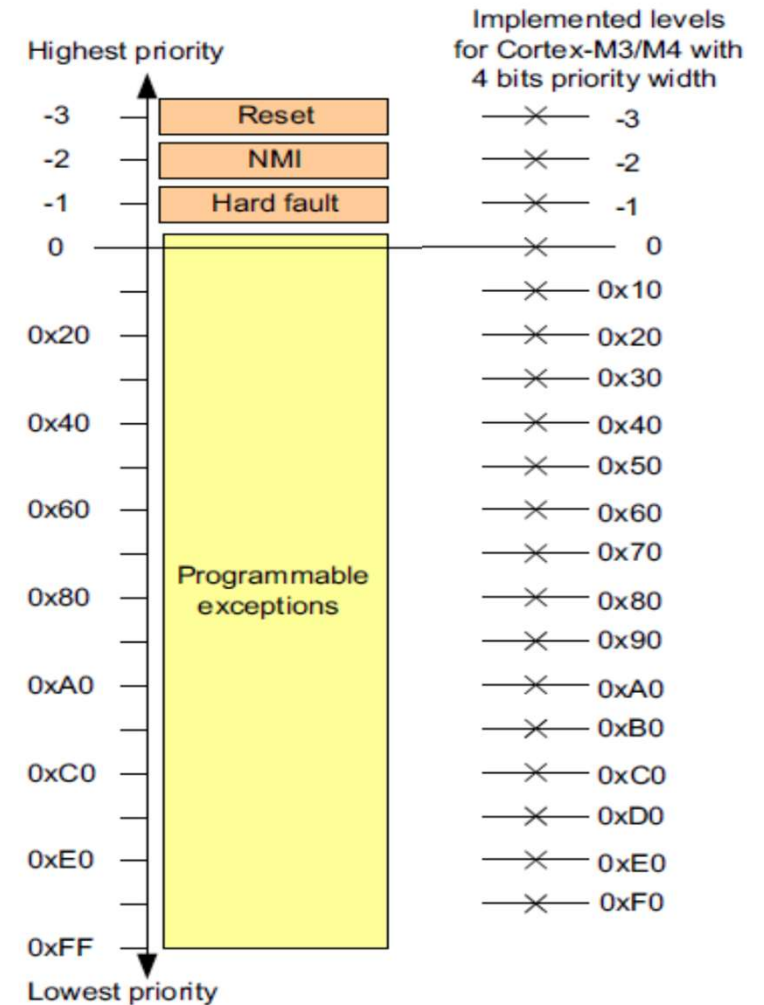
---

Preemption is based on group priority.

Sub-priority is used only for simultaneous requests.

SSP defines 16 priority levels.

| Bit7         | Bit6 | Bit5 | Bit4     | Bit3                     | Bit2 | Bit1 | Bit0 |
|--------------|------|------|----------|--------------------------|------|------|------|
| Preempt Prio |      |      | Sub Prio | Not implemented (read 0) |      |      |      |



# BASEPRI REGISTER

---

## Base Priority Mask Register (BASEPRI)

0 = no effect;

> 0 exceptions with a priority value greater than BASEPRI are not processed. (Rem: greater value means lower priority)

The BASEPRI register defines the minimum priority for exception processing. When BASEPRI is set to a nonzero value, it prevents the activation of all exceptions with the same or lower priority level as the BASEPRI value.